```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>TrafficAI // Command Console</title>
<meta name="theme-color" content="#eef1f5">
<style>
:root {
--asphalt: #f4f6f9;
--panel: #ffffff;
--panel-edge: #e9ecef;
--signal-amber: #0d6efd;
--safety-orange: #dc3545;
--signal-green: #198754;
--text-main: #212529;
--text-dim: #6c757d;
--mono: 'JetBrains Mono', 'Courier New', monospace;
--sans: -apple-system, BlinkMacSystemFont, 'Segoe UI', Inter, sans-serif;
}
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
background: var(--asphalt);
color: var(--text-main);
font-family: var(--sans);
min-height: 100vh;
position: relative;
overflow-x: hidden;
}
.hidden { display: none !important; }
/* scanline motif */
.scanline {
position: fixed; top: 0; left: 0; right: 0; height: 2px;
background: linear-gradient(90deg, transparent, var(--signal-amber), transparent);
opacity: 0.5; animation: scan 6s linear infinite; pointer-events: none; z-index: 50;
}
@keyframes scan { 0% { top: 0%; } 100% { top: 100%; } }
/* ---- Auth screens ---- */
.auth-body { min-height: 100vh; display: flex; align-items: flex-start; justify-content: center; padding: 48px
24px; }
.auth-card { width: 100%; max-width: 420px; background: var(--panel); border: 1px solid var(--panel-edge);
border-radius: 10px; padding: 32px; margin-top: 0; box-shadow: 0 0 10px rgba(0,0,0,0.1); }
.auth-brand { display: flex; align-items: center; gap: 8px; margin-bottom: 6px; }
.auth-brand .brand-dot { width: 9px; height: 9px; }
.auth-title { font-size: 1.2rem; font-weight: 600; margin-bottom: 4px; }
.auth-subtitle { font-size: 0.82rem; color: var(--text-dim); margin-bottom: 24px; line-height: 1.5; }
.field { margin-bottom: 16px; }
.field label { display: block; font-size: 0.72rem; text-transform: uppercase; letter-spacing: 0.06em; color:
var(--text-dim); margin-bottom: 6px; }
.field input, .field select {
width: 100%; background: #f4f6f9; border: 1px solid var(--panel-edge); border-radius: 5px;
padding: 11px 12px; color: var(--text-main); font-family: var(--sans); font-size: 0.92rem;
}
.field input:focus, .field select:focus { outline: none; border-color: var(--signal-amber); }
.readonly-field { background: #f4f6f9; border: 1px solid var(--panel-edge); border-radius: 5px; padding: 11px
12px; color: var(--text-dim); font-family: var(--mono); font-size: 0.9rem; }
.btn-primary { width: 100%; background: var(--signal-amber); color: #ffffff; border: none; border-radius: 6px;
padding: 12px; font-weight: 600; font-size: 0.92rem; cursor: pointer; margin-top: 6px; }
.btn-primary:hover { filter: brightness(1.08); }
.btn-secondary { width: 100%; background: transparent; color: var(--text-dim); border: 1px solid
var(--panel-edge); border-radius: 6px; padding: 11px; font-size: 0.85rem; cursor: pointer; margin-top: 10px; }
.btn-secondary:hover { color: var(--text-main); border-color: var(--text-dim); }
.auth-links { display: flex; justify-content: space-between; margin-top: 18px; font-size: 0.8rem; }
.auth-links a, .auth-links span[role="button"] { color: var(--text-dim); text-decoration: none; cursor: pointer;
}
.auth-links a:hover, .auth-links span[role="button"]:hover { color: var(--signal-amber); }
.alert-box { background: rgba(255,93,58,0.12); border: 1px solid var(--safety-orange); color:
var(--safety-orange); padding: 10px 12px; border-radius: 5px; font-size: 0.82rem; margin-bottom: 16px; }
.success-box { background: rgba(54,211,153,0.12); border: 1px solid var(--signal-green); color:
var(--signal-green); padding: 10px 12px; border-radius: 5px; font-size: 0.82rem; margin-bottom: 16px; }
.info-box { background: rgba(255,176,46,0.1); border: 1px solid var(--signal-amber); color: var(--signal-amber);
padding: 10px 12px; border-radius: 5px; font-size: 0.8rem; margin-bottom: 16px; line-height: 1.5; }
.info-box strong { letter-spacing: 0.03em; }
.demo-accounts { margin-top: 22px; padding-top: 18px; border-top: 1px solid var(--panel-edge); font-size:
0.74rem; color: var(--text-dim); line-height: 1.7; }
.demo-accounts code { color: var(--text-main); font-family: var(--mono); }
.section-title { font-size: 0.78rem; text-transform: uppercase; letter-spacing: 0.08em; color: var(--text-dim);
margin: 22px 0 14px; padding-top: 18px; border-top: 1px solid var(--panel-edge); }
.section-title:first-of-type { border-top: none; padding-top: 0; margin-top: 0; }
/* ---- Dashboard ---- */
.topbar { display: flex; justify-content: space-between; align-items: center; padding: 18px 28px; background:
var(--panel); border-bottom: 1px solid var(--panel-edge); box-shadow: 0 2px 8px rgba(0,0,0,0.05); flex-wrap:
wrap; gap: 12px; }
.brand { display: flex; align-items: center; gap: 10px; }
.brand-dot { width: 10px; height: 10px; border-radius: 50%; background: var(--signal-green); box-shadow: 0 0 8px
var(--signal-green); animation: pulse 2s ease-in-out infinite; }
@keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.35; } }
.brand-name { font-family: var(--mono); font-size: 1.1rem; letter-spacing: 0.08em; }
.brand-name strong { color: var(--signal-amber); }
.brand-sub { font-family: var(--mono); font-size: 0.7rem; color: var(--text-dim); letter-spacing: 0.15em;
margin-left: 6px; }
.clock { font-family: var(--mono); color: var(--text-dim); font-size: 0.95rem; }
.session-info { display: flex; align-items: center; gap: 20px; flex-wrap: wrap; }
.officer-id { text-align: right; font-size: 0.85rem; line-height: 1.4; }
.officer-badge { display: block; font-family: var(--mono); font-size: 0.7rem; color: var(--text-dim); }
.logout-btn { background: transparent; border: 1px solid var(--panel-edge); color: var(--text-dim); font-size:
0.75rem; padding: 7px 12px; border-radius: 4px; cursor: pointer; font-family: var(--sans); }
```

```
.logout-btn:hover { border-color: var(--safety-orange); color: var(--safety-orange); }
.account-link { font-size: 0.75rem; color: var(--text-dim); text-decoration: none; border: 1px solid
var(--panel-edge); padding: 7px 12px; border-radius: 4px; cursor: pointer; background: none; font-family:
var(--sans); }
.account-link:hover { border-color: var(--signal-amber); color: var(--signal-amber); }
.stat-strip { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px; background: var(--asphalt);
border-bottom: none; padding: 18px 18px 0; }
.stat-tile { background: var(--panel); padding: 18px 24px; border-radius: 10px; box-shadow: 0 0 10px
rgba(0,0,0,0.06); }
.stat-label { font-size: 0.72rem; text-transform: uppercase; letter-spacing: 0.1em; color: var(--text-dim);
margin-bottom: 6px; }
.stat-value { font-family: var(--mono); font-size: 1.8rem; font-weight: 600; }
.stat-tile.alert .stat-value { color: var(--safety-orange); }
.stat-tile.ok .stat-value { color: var(--signal-green); }
.grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 14px; background: var(--asphalt); padding: 14px
18px 18px; }
.units-panel { grid-column: 1 / -1; }
.panel { background: var(--panel); display: flex; flex-direction: column; min-height: 320px; border-radius:
10px; box-shadow: 0 0 10px rgba(0,0,0,0.06); overflow: hidden; }
.panel-head { display: flex; justify-content: space-between; align-items: center; padding: 14px 18px;
border-bottom: 1px solid var(--panel-edge); }
.panel-head h2 { font-size: 0.85rem; text-transform: uppercase; letter-spacing: 0.08em; color: var(--text-main);
}
.badge { font-family: var(--mono); font-size: 0.65rem; color: var(--text-dim); border: 1px solid
var(--panel-edge); padding: 3px 8px; border-radius: 3px; }
.badge.live { color: var(--signal-green); border-color: var(--signal-green); }
.badge.alert-badge { color: var(--safety-orange); border-color: var(--safety-orange); }
.feed { flex: 1; overflow-y: auto; max-height: 380px; padding: 8px 0; }
.feed-item { display: flex; justify-content: space-between; gap: 10px; padding: 10px 18px; border-bottom: 1px
solid #e4e8ed; font-size: 0.82rem; animation: fadeIn 0.4s ease; }
@keyframes fadeIn { from { opacity: 0; transform: translateY(-4px); } to { opacity: 1; transform: translateY(0);
} }
.feed-item .main { color: var(--text-main); }
.feed-item .meta { color: var(--text-dim); font-family: var(--mono); font-size: 0.75rem; text-align: right; }
.feed-item.severity-HIGH .main { color: var(--safety-orange); }
.feed-item.severity-MEDIUM .main { color: var(--signal-amber); }
.units { display: grid; grid-template-columns: repeat(auto-fill, minmax(240px, 1fr)); gap: 14px; padding: 18px;
}
.unit-card { border: 1px solid var(--panel-edge); border-radius: 10px; padding: 14px; background: #f7f9fb;
box-shadow: 0 0 8px rgba(0,0,0,0.05); }
.unit-top { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px; }
.unit-name { font-size: 0.85rem; font-weight: 600; }
.unit-status { font-family: var(--mono); font-size: 0.68rem; padding: 2px 7px; border-radius: 3px;
text-transform: uppercase; }
.unit-status.ONLINE { background: rgba(54,211,153,0.15); color: var(--signal-green); }
.unit-status.OFFLINE { background: rgba(255,93,58,0.15); color: var(--safety-orange); }
.unit-status.SYNCING { background: rgba(255,176,46,0.15); color: var(--signal-amber); }
.unit-meta { font-size: 0.75rem; color: var(--text-dim); line-height: 1.6; }
.unit-meta span { color: var(--text-main); }
.battery-bar { height: 4px; background: #e4e8ed; border-radius: 2px; margin-top: 8px; overflow: hidden; }
.battery-fill { height: 100%; background: var(--signal-green); }
.activity-badge { margin-top: 8px; font-family: var(--mono); font-size: 0.66rem; text-transform: uppercase;
letter-spacing: 0.04em; padding: 3px 8px; border-radius: 3px; display: inline-block; }
.activity-ON_DUTY { background: rgba(54,211,153,0.15); color: var(--signal-green); }
.activity-PHONE_DISTRACTION { background: rgba(255,93,58,0.15); color: var(--safety-orange); }
.activity-OFF_DUTY { background: rgba(134,148,163,0.15); color: var(--text-dim); }
.activity-NOT_STANDING { background: rgba(255,176,46,0.15); color: var(--signal-amber); }
.duty-INAPPROPRIATE_LANGUAGE .main { color: var(--safety-orange); font-weight: 600; }
.storage-note { padding: 0 18px 18px; font-size: 0.75rem; color: var(--text-dim); line-height: 1.6; }
.storage-note strong { color: var(--text-main); }
.trend-panel { grid-column: 1 / -1; }
.legend-dot { display: inline-block; width: 7px; height: 7px; border-radius: 50%; margin: 0 4px 0 8px;
vertical-align: middle; }
.legend-dot:first-child { margin-left: 0; }
.legend-dot.vehicles { background: var(--signal-green); }
.legend-dot.violations { background: var(--safety-orange); }
.trend-chart { display: flex; gap: 18px; padding: 24px 20px 18px; align-items: flex-end; height: 220px;
overflow-x: auto; }
.trend-day { flex: 1; min-width: 34px; display: flex; flex-direction: column; align-items: center; gap: 8px;
height: 100%; justify-content: flex-end; }
.trend-bars { display: flex; gap: 5px; align-items: flex-end; height: 160px; }
.trend-bar { width: 16px; border-radius: 2px 2px 0 0; min-height: 2px; transition: height 0.3s ease; }
.trend-bar.vehicles { background: var(--signal-green); }
.trend-bar.violations { background: var(--safety-orange); }
.trend-label { font-size: 0.68rem; color: var(--text-dim); font-family: var(--mono); }
.trend-label.today { color: var(--signal-amber); }
.offenders-panel { grid-column: 1 / -1; }
.offenders-table { padding: 6px 18px 18px; }
.offender-row { display: grid; grid-template-columns: 1.2fr 1fr 1fr 1fr; gap: 10px; align-items: center;
padding: 10px 0; border-bottom: 1px solid #e4e8ed; font-size: 0.82rem; }
.offender-row.head { color: var(--text-dim); font-size: 0.68rem; text-transform: uppercase; letter-spacing:
0.06em; border-bottom: 1px solid var(--panel-edge); }
.offender-plate { font-family: var(--mono); }
.offender-row.repeat .offender-plate { color: var(--safety-orange); }
.repeat-tag { display: inline-block; font-family: var(--mono); font-size: 0.62rem; background:
rgba(255,93,58,0.15); color: var(--safety-orange); padding: 2px 6px; border-radius: 3px; margin-left: 8px; }
.empty-note { color: var(--text-dim); font-size: 0.8rem; padding: 12px 0; }
.feed-item.violation { flex-direction: column; align-items: stretch; gap: 4px; }
.feed-item.violation .meta { text-align: left; }
.feed-item .fine-row { display: flex; justify-content: space-between; align-items: center; margin-top: 4px; }
.fine-amount { font-family: var(--mono); font-size: 0.78rem; color: var(--text-main); }
.fine-amount.repeat { color: var(--safety-orange); font-weight: 600; }
.pay-btn { font-family: var(--sans); font-size: 0.68rem; background: var(--signal-amber); color: #ffffff;
border: none; border-radius: 3px; padding: 4px 10px; cursor: pointer; font-weight: 600; }
.pay-btn:hover { filter: brightness(1.08); }
.paid-tag { font-family: var(--mono); font-size: 0.68rem; color: var(--signal-green); }
.manual-fine-panel { padding: 14px 18px; border-bottom: 1px solid var(--panel-edge); background: #f7f9fb; }
```

```
.manual-fine-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 10px;
margin-bottom: 10px; }
.manual-fine-grid .field { margin-bottom: 0; }
.manual-fine-grid .field input, .manual-fine-grid .field select {
width: 100%; background: #f4f6f9; border: 1px solid var(--panel-edge); border-radius: 5px;
padding: 9px 10px; color: var(--text-main); font-family: var(--sans); font-size: 0.82rem;
}
.manual-fine-grid .field input:focus, .manual-fine-grid .field select:focus { outline: none; border-color:
var(--signal-amber); }
.manual-fine-actions { display: flex; gap: 10px; align-items: center; }
.manual-fine-actions .btn-primary, .manual-fine-actions .btn-secondary { width: auto; padding: 9px 16px;
margin-top: 0; font-size: 0.8rem; }
#manual-fine-msg { font-size: 0.76rem; margin-top: 8px; }
#manual-fine-msg.error { color: var(--safety-orange); }
#manual-fine-msg.success { color: var(--signal-green); }
.duty-panel { grid-column: 1 / -1; }
.officer-roster { padding: 4px 0 6px; }
.officer-roster-pair { padding: 10px 0; border-bottom: 1px solid var(--panel-edge); }
.officer-roster-pair:last-child { border-bottom: none; }
.officer-roster-row { display: flex; justify-content: space-between; align-items: center; gap: 12px; flex-wrap:
wrap; }
.officer-roster-row.duty-line { margin-top: 8px; padding-top: 8px; border-top: 1px dashed var(--panel-edge); }
.officer-roster-info { font-size: 0.82rem; color: var(--text-main); }
.officer-roster-badge { font-family: var(--mono); font-size: 0.7rem; color: var(--text-dim); margin-left: 6px; }
.officer-roster-meta { font-size: 0.75rem; color: var(--text-dim); margin-top: 2px; }
.officer-roster-controls { display: flex; gap: 8px; align-items: center; }
.officer-roster-controls select { background: var(--panel); border: 1px solid var(--panel-edge); border-radius:
5px; padding: 6px 8px; font-size: 0.78rem; color: var(--text-main); font-family: var(--sans); }
#add-officer-msg { font-size: 0.76rem; margin-top: 8px; }
#add-officer-msg.error { color: var(--safety-orange); }
#add-officer-msg.success { color: var(--signal-green); }
/* ---- Camera switching select ---- */
.camera-select { font-family: var(--mono); font-size: 0.68rem; background: var(--panel); border: 1px solid
var(--panel-edge); border-radius: 3px; padding: 5px 8px; color: var(--text-main); cursor: pointer; }
/* ---- AI check toggles (overspeed / helmet) + photo upload ---- */
.ai-toggle-row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; padding: 8px 18px;
border-bottom: 1px solid var(--panel-edge); background: #f7f9fb; }
.ai-toggle { display: inline-flex; align-items: center; gap: 5px; font-family: var(--mono); font-size: 0.68rem;
color: var(--text-dim); cursor: pointer; user-select: none; }
.ai-toggle input[type="checkbox"] { accent-color: var(--signal-amber); width: 13px; height: 13px; cursor:
pointer; }
.ai-toggle.off { color: var(--safety-orange); text-decoration: line-through; opacity: 0.75; }
.upload-btn { font-family: var(--sans); font-size: 0.7rem; }
.photo-status-line { font-size: 0.72rem; color: var(--text-dim); margin-top: 6px; font-family: var(--mono); }
.checklist-line { font-size: 0.7rem; color: var(--text-main); margin-top: 6px; font-family: var(--mono);
line-height: 1.6; }
.checklist-line .pass { color: #2f9e5c; }
.checklist-line .fail { color: #dc3545; font-weight: 600; }
.checklist-line .skip { color: var(--text-dim); font-style: italic; }
.offenders-panel .panel-head { flex-wrap: wrap; gap: 6px; }
.repeat-edit-form { display: inline-flex; align-items: center; gap: 5px; font-family: var(--mono); font-size:
0.68rem; color: var(--text-dim); }
.repeat-edit-form input[type="number"] { padding: 2px 4px; border: 1px solid var(--panel-edge); border-radius:
3px; }
@keyframes flashHighlight {
0% { background: rgba(220,53,69,0.22); }
100% { background: transparent; }
}
.flash-highlight { animation: flashHighlight 2.2s ease-out; border-radius: 6px; }
/* ---- Unit online/offline toggle ---- */
.unit-card-actions { margin-top: 10px; }
.status-toggle-btn { font-family: var(--sans); font-size: 0.68rem; border: 1px solid var(--panel-edge);
background: transparent; color: var(--text-dim); border-radius: 3px; padding: 5px 10px; cursor: pointer;
font-weight: 600; }
.status-toggle-btn:hover { border-color: var(--signal-amber); color: var(--signal-amber); }
.status-toggle-btn.is-online { color: var(--safety-orange); }
.status-toggle-btn.is-online:hover { border-color: var(--safety-orange); color: var(--safety-orange); }
.status-toggle-btn.is-offline { color: var(--signal-green); }
.status-toggle-btn.is-offline:hover { border-color: var(--signal-green); color: var(--signal-green); }
/* ---- Officer-online duty alert ---- */
.duty-OFFICER_ONLINE .main { color: var(--signal-green); font-weight: 600; }
.duty-PHONE_DISTRACTION .main { color: var(--safety-orange); font-weight: 600; }
.duty-NOT_STANDING .main { color: var(--safety-orange); font-weight: 600; }
.duty-BEHAVIOR_CHECK_CLEAR .main { color: var(--signal-green); font-weight: 600; }
/* ---- Duty type control + behavior check ---- */
.duty-type-badge { font-family: var(--mono); font-size: 0.66rem; text-transform: uppercase; letter-spacing:
0.04em; padding: 3px 8px; border-radius: 3px; display: inline-block; background: rgba(13,110,253,0.12); color:
var(--signal-amber); margin-top: 6px; }
.check-btn { font-family: var(--sans); font-size: 0.68rem; border: 1px solid var(--panel-edge); background:
transparent; color: var(--text-dim); border-radius: 3px; padding: 5px 10px; cursor: pointer; font-weight: 600; }
.check-btn:hover { border-color: var(--signal-amber); color: var(--signal-amber); }
.check-btn:disabled { opacity: 0.45; cursor: not-allowed; }
.check-btn:disabled:hover { border-color: var(--panel-edge); color: var(--text-dim); }
.officer-roster-controls-row { display: flex; justify-content: space-between; align-items: center; gap: 12px;
flex-wrap: wrap; padding: 6px 0; }
.officer-roster-controls-row:not(:last-child) { border-bottom: 1px dashed var(--panel-edge); }
.officer-roster-controls-row .officer-roster-controls select { min-width: 130px; }
/* ---- Toast notifications ---- */
.toast-container { position: fixed; top: 76px; right: 20px; z-index: 500; display: flex; flex-direction: column;
gap: 10px; max-width: 320px; }
.toast { background: var(--panel); border: 1px solid var(--panel-edge); border-left: 4px solid
var(--signal-green); border-radius: 6px; padding: 12px 16px; box-shadow: 0 6px 16px rgba(0,0,0,0.18); font-size:
0.82rem; color: var(--text-main); animation: toastIn 0.25s ease; }
.toast.offline-toast { border-left-color: var(--safety-orange); }
@keyframes toastIn { from { opacity: 0; transform: translateX(24px); } to { opacity: 1; transform:
translateX(0); } }
@keyframes toastOut { from { opacity: 1; } to { opacity: 0; } }
```

```
.footer-note { text-align: center; padding: 18px; font-size: 0.72rem; color: var(--text-dim); }
/* ---- Site navbar (matches sample's dark navbar) ---- */
.site-navbar { background: #212529; color: #fff; padding: 14px 28px; display: flex; align-items: center;
justify-content: space-between; flex-wrap: wrap; gap: 12px; }
.site-navbar .brand-name { color: #fff; }
.site-navbar .brand-name strong { color: #5b9dff; }
.site-navbar .nav-meta { display: flex; align-items: center; gap: 16px; flex-wrap: wrap; }
.site-navbar .clock { color: #adb5bd; }
.site-navbar .officer-id { color: #e9ecef; }
.site-navbar .officer-badge { color: #adb5bd; }
.site-navbar .logout-btn, .site-navbar .account-link { border-color: rgba(255,255,255,0.25); color: #e9ecef; }
.site-navbar .logout-btn:hover, .site-navbar .account-link:hover { border-color: #fff; color: #fff; }
/* ---- Hero banner (matches sample's blue hero) ---- */
.login-hero { background: var(--signal-amber); color: #fff; padding: 56px 20px; text-align: center; }
.login-hero h1 { font-size: 1.7rem; font-weight: 700; margin-bottom: 10px; }
.login-hero p { opacity: 0.92; font-size: 0.92rem; max-width: 520px; margin: 0 auto; line-height: 1.5; }
/* ---- Card hover-lift (matches sample's .feature-card:hover) ---- */
.panel, .stat-tile, .unit-card { transition: transform 0.3s, box-shadow 0.3s; }
.panel:hover, .stat-tile:hover, .unit-card:hover { transform: translateY(-4px); box-shadow: 0 6px 16px
rgba(0,0,0,0.1); }
/* ---- Dark footer (matches sample's footer) ---- */
.site-footer { background: #111; color: #fff; padding: 20px; text-align: center; font-size: 0.78rem; }
.site-footer a, .site-footer span[role="button"] { color: #adb5bd; cursor: pointer; text-decoration: underline;
}
@media (max-width: 900px) {
.grid { grid-template-columns: 1fr; }
.stat-strip { grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 600px) {
/* Bigger touch targets for the camera/photo/video toolbar on phones */
.feed-panel .panel-head { flex-direction: column; align-items: stretch; }
.feed-panel .panel-head > div { width: 100%; }
.feed-panel .panel-head .account-link,
.feed-panel .panel-head .camera-select { flex: 1 1 auto; min-height: 40px; font-size: 0.74rem; padding: 9px
10px; touch-action: manipulation; }
.ai-toggle-row { padding: 10px 14px; }
.ai-toggle { font-size: 0.74rem; }
.ai-toggle input[type="checkbox"] { width: 18px; height: 18px; }
/* Let the camera/photo/video preview fill the available width instead of capping at 360px */
#camera-wrap > div, #photo-wrap > div, #uploaded-video-wrap > div { max-width: 100% !important; }
.manual-fine-grid { grid-template-columns: 1fr; }
.repeat-edit-form { flex-wrap: wrap; }
}
</style>
</head>
<body>
<div class="scanline"></div>
<div class="toast-container" id="toast-container"></div>
<!-- ============ LOGIN VIEW ============ -->
<div id="login-view">
<nav class="site-navbar">
<span class="brand-name">□ TRAFFIC<strong>AI</strong></span>
</nav>
<div class="login-hero">
<h1>AI Traffic Violation Detection System</h1>
<p>Detect traffic violations automatically using AI, CCTV cameras and number-plate recognition. Officer sign-in
below.</p>
</div>
<div class="auth-body">
<div class="auth-card">
<div class="auth-brand">
<span class="brand-dot"></span>
<span class="brand-name">TRAFFIC<strong>AI</strong></span>
</div>
<div class="auth-title">Officer Sign In</div>
<div class="auth-subtitle">Authorized personnel only. Sign in with your badge ID and password.</div>
<div id="login-error" class="alert-box hidden">Incorrect Badge ID or password. Please try again.</div>
<div id="login-success" class="success-box hidden"></div>
<form id="login-form">
<div class="field">
<label for="badgeId">Badge ID / Government Email</label>
<input type="text" id="badgeId" placeholder="e.g. DL-1042" autocomplete="username" required>
</div>
<div class="field">
<label for="password">Password</label>
<input type="password" id="password" placeholder="••••••••" autocomplete="current-password" required>
</div>
<button type="submit" class="btn-primary">Sign In</button>
</form>
<div class="auth-links">
<span role="button" tabindex="0" id="forgot-link">Forgot password?</span>
<span role="button" tabindex="0" id="register-link">Create an account</span>
</div>
<div class="demo-accounts">
<strong>Demo accounts</strong> (this is a fully simulated, offline demo — no real data leaves your browser):<br>
Badge <code>DL-1042</code> / Password <code>Patrol@123</code><br>
Badge <code>DL-2089</code> / Password <code>Traffic@456</code><br>
Badge <code>ADM-0001</code> / Password <code>Command@789</code> (admin)<br>
Badge <code>suyash</code> / Password <code>suyash@123</code>
</div>
</div>
</div>
<footer class="site-footer">TrafficAI — simulated demo. No real data leaves your browser.</footer>
</div>
<!-- ============ REGISTER VIEW ============ -->
<div id="register-view" class="hidden">
<nav class="site-navbar">
```

```
<span class="brand-name">🚦 TRAFFIC<strong>AI</strong></span>
</nav>
<div class="auth-body">
<div class="auth-card">
<div class="auth-brand"><span class="brand-dot"></span><span
class="brand-name">TRAFFIC<strong>AI</strong></span></div>
<div class="auth-title">Create Officer Account</div>
<div class="auth-subtitle">Register a new badge ID for this simulated demo. Everything stays saved only in this
browser.</div>
<div id="register-error" class="alert-box hidden"></div>
<div id="register-success" class="success-box hidden"></div>
<form id="register-form">
<div class="field"><label for="regBadgeId">Badge ID</label><input type="text" id="regBadgeId" placeholder="e.g.
DL-3051" autocomplete="off" required></div>
<div class="field"><label for="regFullName">Full Name</label><input type="text" id="regFullName"
required></div>
<div class="field"><label for="regRank">Rank</label>
<select id="regRank" required>
<option value="Constable">Constable</option>
<option value="Sub-Inspector">Sub-Inspector</option>
<option value="Inspector">Inspector</option>
<option value="Inspector (Admin)">Inspector (Admin)</option>
</select>
</div>
<div class="field"><label for="regStation">Station</label><input type="text" id="regStation" placeholder="e.g.
City Center Patrol" required></div>
<div class="field"><label for="regGovEmail">Government Email</label><input type="email" id="regGovEmail"
required></div>
<div class="field"><label for="regPhoneNumber">Phone Number</label><input type="tel" id="regPhoneNumber"
placeholder="+91XXXXXXXXXX" required></div>
<div class="field"><label for="regPassword1">Password</label><input type="password" id="regPassword1"
placeholder="At least 8 characters" autocomplete="new-password" required></div>
<div class="field"><label for="regPassword2">Confirm Password</label><input type="password" id="regPassword2"
autocomplete="new-password" required></div>
<button type="submit" class="btn-primary">Create Account</button>
</form>
<div class="auth-links"><span role="button" tabindex="0" id="back-to-login-from-register">&larr; Back to sign
in</span><span></span></div>
</div>
</div>
</div>
<!-- ============ FORGOT PASSWORD VIEW ============ -->
<div id="forgot-view" class="hidden">
<nav class="site-navbar">
<span class="brand-name">🚦 TRAFFIC<strong>AI</strong></span>
</nav>
<div class="auth-body">
<div class="auth-card">
<div class="auth-brand"><span class="brand-dot"></span><span
class="brand-name">TRAFFIC<strong>AI</strong></span></div>
<div class="auth-title">Reset Password</div>
<div class="auth-subtitle">Enter your Badge ID. A one-time code will be "sent" to your registered phone
(simulated — shown on screen for this demo).</div>
<div id="forgot-error" class="alert-box hidden"></div>
<div id="forgot-otp-box" class="info-box hidden"></div>
<form id="forgot-step1-form">
<div class="field">
<label for="forgotBadgeId">Badge ID</label>
<input type="text" id="forgotBadgeId" placeholder="e.g. DL-1042" required>
</div>
<button type="submit" class="btn-primary">Send Reset Code</button>
</form>
<form id="forgot-step2-form" class="hidden">
<div class="field">
<label for="otpInput">One-Time Code</label>
<input type="text" id="otpInput" placeholder="6-digit code" required>
</div>
<div class="field">
<label for="newPassword1">New Password</label>
<input type="password" id="newPassword1" placeholder="At least 8 characters" required>
</div>
<div class="field">
<label for="confirmPassword1">Confirm New Password</label>
<input type="password" id="confirmPassword1" required>
</div>
<button type="submit" class="btn-primary">Update Password</button>
</form>
<div class="auth-links"><span role="button" tabindex="0" id="back-to-login">&larr; Back to sign
in</span><span></span></div>
</div>
</div>
</div>
<!-- ============ ACCOUNT VIEW ============ -->
<div id="account-view" class="hidden">
<nav class="site-navbar">
<span class="brand-name">🚦 TRAFFIC<strong>AI</strong></span>
</nav>
<div class="auth-body">
<div class="auth-card">
<div class="auth-brand"><span class="brand-dot"></span><span
class="brand-name">TRAFFIC<strong>AI</strong></span></div>
<div class="auth-title">Account Settings</div>
<div class="auth-subtitle">Update your contact details or password.</div>
<div class="section-title">Badge</div>
<div class="readonly-field" id="account-badge-info"></div>
<div class="section-title">Contact Details</div>
<div id="contact-success" class="success-box hidden">Contact details updated.</div>
```

```
<form id="contact-form">
<div class="field"><label for="govEmail">Government Email</label><input type="email" id="govEmail"
required></div>
<div class="field"><label for="phoneNumber">Registered Phone Number</label><input type="tel" id="phoneNumber"
placeholder="+91XXXXXXXXXX" required></div>
<button type="submit" class="btn-primary">Save Contact Details</button>
</form>
<div class="section-title">Change Password</div>
<div id="password-success" class="success-box hidden">Password updated.</div>
<div id="password-error" class="alert-box hidden"></div>
<form id="password-form">
<div class="field"><label for="currentPassword">Current Password</label><input type="password"
id="currentPassword" required></div>
<div class="field"><label for="newPassword2">New Password</label><input type="password" id="newPassword2"
placeholder="At least 8 characters" required></div>
<div class="field"><label for="confirmPassword2">Confirm New Password</label><input type="password"
id="confirmPassword2" required></div>
<button type="submit" class="btn-primary">Update Password</button>
</form>
<div class="auth-links"><span role="button" tabindex="0" id="back-to-dashboard">&larr; Back to
dashboard</span><span></span></div>
</div>
</div>
</div>
<!-- ============ DASHBOARD VIEW ============ -->
<div id="dashboard-view" class="hidden">
<header class="topbar site-navbar">
<div class="brand">
<span class="brand-dot"></span>
<span class="brand-name">🚦 TRAFFIC<strong>AI</strong></span>
<span class="brand-sub">COMMAND CONSOLE</span>
</div>
<div class="session-info nav-meta">
<div class="officer-id">
<span id="officer-name">Officer Name</span>
<span class="officer-badge" id="officer-badge-rank">Badge · Rank</span>
</div>
<div class="clock" id="clock">--:--:--</div>
<button class="account-link" id="account-btn">Account</button>
<button class="logout-btn" id="logout-btn">Sign out</button>
</div>
</header>
<section class="stat-strip">
<div class="stat-tile"><div class="stat-label">Vehicles Detected</div><div class="stat-value"
id="stat-vehicles">0</div></div>
<div class="stat-tile alert"><div class="stat-label">Violations Flagged</div><div class="stat-value"
id="stat-violations">0</div></div>
<div class="stat-tile ok"><div class="stat-label">Units Online</div><div class="stat-value" id="stat-units">0 /
0</div></div>
<div class="stat-tile"><div class="stat-label">Plates Read</div><div class="stat-value"
id="stat-plates">0</div></div>
</section>
<main class="grid">
<section class="panel feed-panel">
<div class="panel-head">
<h2>Vehicle Stream</h2>
<div style="display:flex;gap:8px;align-items:center;flex-wrap:wrap;">
<select id="camera-select" class="camera-select" title="Assign detections to this simulated unit"></select>
<select id="camera-device-select" class="camera-select" title="Pick which physical camera (built-in or
external/USB/phone) to use"></select>
<button class="account-link" id="camera-flip-btn" style="cursor:pointer;" title="Switch between front and back
camera">⇄ Switch to Front</button>
<span class="badge" id="source-badge">SIMULATED</span>
<button class="account-link" id="camera-toggle-btn" style="cursor:pointer;">Enable Camera AI</button>
<input type="file" id="photo-upload-input" accept="image/*" class="hidden">
<button class="account-link upload-btn" id="photo-upload-btn" style="cursor:pointer;">Upload Photo</button>
<input type="file" id="video-upload-input" accept="video/*" class="hidden">
<button class="account-link upload-btn" id="video-upload-btn" style="cursor:pointer;">Upload Video</button>
<button class="account-link" id="camera-manual-add-btn" style="cursor:pointer;">+ Add Violation
Manually</button>
</div>
</div>
<div class="ai-toggle-row">
<label class="ai-toggle" id="toggle-overspeed-label" for="toggle-overspeed">
<input type="checkbox" id="toggle-overspeed" checked> Overspeed AI
</label>
<label class="ai-toggle" id="toggle-helmet-label" for="toggle-helmet">
<input type="checkbox" id="toggle-helmet" checked> Helmet AI
</label>
<span style="font-size:0.66rem;color:var(--text-dim);font-family:var(--mono);">applies to live camera &amp;
uploaded photos</span>
</div>
<div id="camera-wrap" class="hidden" style="padding:10px 18px;border-bottom:1px solid var(--panel-edge);">
<div style="position:relative;width:100%;max-width:360px;min-height:200px;background:#000;border-radius:6px;ove
rflow:hidden;">
<video id="camera-video" autoplay muted playsinline webkit-playsinline="true"
style="width:100%;height:100%;object-fit:cover;display:block;"></video>
<canvas id="camera-canvas"
style="position:absolute;top:0;left:0;width:100%;height:100%;pointer-events:none;"></canvas>
</div>
<div id="camera-status"
style="font-size:0.72rem;color:var(--text-dim);margin-top:6px;font-family:var(--mono);">Loading model…</div>
<div id="camera-checklist" class="checklist-line"></div>
</div>
<div id="photo-wrap" class="hidden" style="padding:10px 18px;border-bottom:1px solid var(--panel-edge);">
<div style="position:relative;width:100%;max-width:360px;min-height:200px;background:#000;border-radius:6px;ove
rflow:hidden;">
```

```
<img id="photo-img" style="width:100%;height:100%;object-fit:contain;display:block;" alt="Uploaded photo for AI
review">
<canvas id="photo-canvas"
style="position:absolute;top:0;left:0;width:100%;height:100%;pointer-events:none;"></canvas>
</div>
<div id="photo-status" class="photo-status-line"></div>
<div id="photo-checklist" class="checklist-line"></div>
</div>
<div id="uploaded-video-wrap" class="hidden" style="padding:10px 18px;border-bottom:1px solid
var(--panel-edge);">
<div style="position:relative;width:100%;max-width:360px;min-height:200px;background:#000;border-radius:6px;ove
rflow:hidden;">
<video id="uploaded-video" controls playsinline webkit-playsinline="true"
style="width:100%;height:100%;object-fit:contain;display:block;"></video>
<canvas id="uploaded-video-canvas"
style="position:absolute;top:0;left:0;width:100%;height:100%;pointer-events:none;"></canvas>
</div>
<div id="uploaded-video-status" class="photo-status-line"></div>
<div id="uploaded-video-checklist" class="checklist-line"></div>
</div>
<div class="feed" id="vehicle-feed"></div>
</section>
<section class="panel feed-panel">
<div class="panel-head">
<h2>Violation Alerts</h2>
<div style="display:flex;gap:8px;align-items:center;">
<span class="badge live alert-badge">LIVE</span>
<button class="account-link" id="manual-fine-toggle-btn" style="cursor:pointer;">Issue Fine</button>
</div>
</div>
<div id="manual-fine-panel" class="manual-fine-panel hidden">
<form id="manual-fine-form">
<div class="manual-fine-grid">
<div class="field">
<label for="mf-plate">Plate Number</label>
<input type="text" id="mf-plate" placeholder="e.g. DL01AB1234" autocomplete="off" required>
</div>
<div class="field">
<label for="mf-vehicle-type">Vehicle Type</label>
<select id="mf-vehicle-type"></select>
</div>
<div class="field">
<label for="mf-violation-type">Violation</label>
<select id="mf-violation-type"></select>
</div>
<div class="field">
<label for="mf-location">Location</label>
<select id="mf-location"></select>
</div>
</div>
<div class="manual-fine-actions">
<button type="submit" class="btn-primary">Issue Fine</button>
<button type="button" class="btn-secondary" id="manual-fine-cancel-btn">Cancel</button>
</div>
<div id="manual-fine-msg"></div>
</form>
</div>
<div class="feed" id="violation-feed"></div>
</section>
<section class="panel feed-panel">
<div class="panel-head"><h2>Plate Recognition</h2><span class="badge live">LIVE</span></div>
<div class="feed" id="plate-feed"></div>
</section>
<section class="panel units-panel">
<div class="panel-head"><h2>Field Units</h2><span class="badge">Fixed Cams &amp;amp; Officer Cams</span></div>
<div class="units" id="units-list"></div>
<p class="storage-note">
All confirmed footage syncs to <strong>Secure Gov Drive</strong> over wired or Bluetooth links.
This is a simulated demo — wiring it to real cameras requires legal authorization,
encryption in transit/at rest, and a data-retention policy.
</p>
</section>
<section class="panel duty-panel">
<div class="panel-head">
<h2>Officer Duty Monitor</h2>
<div style="display:flex;gap:8px;align-items:center;">
<span class="badge live alert-badge">LIVE</span>
<button class="account-link" id="officer-manage-toggle-btn" style="cursor:pointer;">Manage Officers</button>
</div>
</div>
<div id="officer-manage-panel" class="manual-fine-panel hidden">
<div class="section-title" style="margin-top:0;padding-top:0;border-top:none;">Add Officer To Patrol</div>
<form id="add-officer-form">
<div class="manual-fine-grid">
<div class="field">
<label for="off-name">Officer Name</label>
<input type="text" id="off-name" placeholder="e.g. Meera Nair" required>
</div>
<div class="field">
<label for="off-badge">Badge ID</label>
<input type="text" id="off-badge" placeholder="e.g. DL-4021" autocomplete="off" required>
</div>
<div class="field">
<label for="off-street">Assign Street</label>
<select id="off-street"></select>
</div>
</div>
```

```
<div class="manual-fine-actions">
<button type="submit" class="btn-primary">Add Officer</button>
</div>
<div id="add-officer-msg"></div>
</form>
<div class="section-title">Reassign Officer To Another Street</div>
<div class="officer-roster" id="officer-roster"></div>
</div>
<div class="feed" id="duty-feed"></div>
<p class="storage-note">
Detects phone use and non-standing posture (from video), plus abusive or unparliamentary
language (from audio). These are simulated signals. Real posture/phone detection needs
pose-estimation + object-detection on the video; real language detection needs speech-to-text
plus a reviewed word list, and always needs a human reviewing flagged clips before any
disciplinary action. Audio recording is legally more sensitive than video in most places —
confirm consent/authorization requirements before capturing any real audio.
</p>
</section>
<section class="panel offenders-panel">
<div class="panel-head">
<h2>Repeat Offenders</h2>
<span class="badge alert-badge" id="repeat-badge-display">&gt; 7 violations = 3&times; fine</span>
<button class="account-link" id="repeat-edit-btn" style="cursor:pointer;font-size:0.68rem;">Edit</button>
<span class="repeat-edit-form hidden" id="repeat-edit-form">
&gt; <input type="number" id="repeat-threshold-input" min="1" step="1"
style="width:48px;font-family:var(--mono);font-size:0.68rem;">
violations = <input type="number" id="repeat-multiplier-input" min="1" step="0.1"
style="width:48px;font-family:var(--mono);font-size:0.68rem;">&times; fine
<button class="account-link" id="repeat-save-btn" style="cursor:pointer;">Save</button>
<button class="account-link" id="repeat-cancel-btn" style="cursor:pointer;">Cancel</button>
</span>
</div>
<div class="offenders-table" id="offenders-table"></div>
</section>
<section class="panel offenders-panel">
<div class="panel-head">
<h2>Fine Amounts (₹)</h2>
<span class="badge" style="font-size:0.62rem;">per violation type, before repeat-offender multiplier</span>
<button class="account-link" id="fine-amounts-toggle-btn" style="cursor:pointer;">Edit</button>
</div>
<div id="fine-amounts-panel" class="manual-fine-panel hidden">
<div class="manual-fine-grid" id="fine-amounts-grid"></div>
<div class="manual-fine-actions">
<button type="button" class="btn-primary" id="fine-amounts-save-btn">Save</button>
<button type="button" class="btn-secondary" id="fine-amounts-reset-btn">Reset to Defaults</button>
<button type="button" class="btn-secondary" id="fine-amounts-cancel-btn">Cancel</button>
</div>
<div id="fine-amounts-msg" style="font-size:0.76rem;margin-top:8px;"></div>
</div>
</section>
<section class="panel trend-panel">
<div class="panel-head">
<h2>7-Day Trend</h2>
<span class="badge"><span class="legend-dot vehicles"></span> Vehicles<span class="legend-dot
violations"></span> Violations</span>
</div>
<div class="trend-chart" id="trend-chart"></div>
</section>
</main>
<div class="footer-note site-footer">
Vehicle/person detection can run on your real camera (built-in, external/USB, or phone — pick it from the camera
dropdown), an uploaded photo, or an uploaded video with continuous live detection, all using an in-browser AI
model (bounding boxes and object classes are genuine model output). For every detected vehicle, the app runs the
full violation checklist for its type (helmet, license, number plate, seatbelt, emissions, vehicle condition,
wrong lane, signal jump, overspeed, alcohol, noise) and shows each check as TRUE (pass) or FALSE (violation).
This demo has no real speed radar, breathalyzer, or helmet-shape classifier, so each individual pass/fail call
is simulated, not measured — the Overspeed AI / Helmet AI toggles let you force those two specific checks off.
You can also add a violation by hand any time with "+ Add Violation Manually" or "Issue Fine." Plate numbers,
alcohol level, and noise readings also remain simulated. The "&gt; N violations = X&times; fine" repeat-offender
rule is editable via the Edit button on the Repeat Offenders panel.
Data (violations, fines, accounts) is saved to this browser's local storage so it survives closing and reopening
the file. <span role="button" tabindex="0" id="reset-data-link">Reset all saved data</span>
</div>
</div>
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@4.20.0/dist/tf.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/coco-ssd@2.2.3/dist/coco-ssd.min.js"></script>
<script>
/* ============================================================
TrafficAI — standalone static build
Everything below runs entirely client-side in the browser.
It mirrors the logic of the original Spring Boot backend
(TrafficSimulationService) but generates and stores all
data locally — nothing is sent anywhere.
============================================================ */
// ---------- Officer directory (demo accounts) ----------
const OFFICERS = {
'DL-1042': { badgeId: 'DL-1042', fullName: 'Ranjit Kumar', rank: 'Sub-Inspector', station: 'Main St Traffic
Post', govEmail: 'ranjit.kumar@police.gov.in', phoneNumber: '+919812345231', password: 'Patrol@123' },
'DL-2089': { badgeId: 'DL-2089', fullName: 'Asha Verma', rank: 'Constable', station: 'City Center Patrol',
govEmail: 'asha.verma@police.gov.in', phoneNumber: '+919876543210', password: 'Traffic@456' },
'ADM-0001': { badgeId: 'ADM-0001', fullName: 'Vikram Singh', rank: 'Inspector (Admin)', station: 'Command
Center', govEmail: 'vikram.singh@police.gov.in', phoneNumber: '+919900112233', password: 'Command@789' },
'suyash': { badgeId: 'suyash', fullName: 'Suyash', rank: 'Inspector (Admin)', station: 'Command Center',
govEmail: 'suyash@police.gov.in', phoneNumber: '+910000000000', password: 'suyash@123' },
};
let currentOfficer = null;
let pendingOtp = null;
```

```
let pendingOtpBadge = null;
function maskPhone(phone) {
if (!phone || phone.length < 3) return phone;
const last3 = phone.slice(-3);
return '•'.repeat(Math.max(0, phone.length - 3)) + last3;
}
// ---------- Persistence (real browser localStorage — survives closing/reopening this file) ----------
const LS_STATE_KEY = 'trafficai_state_v1';
const LS_OFFICERS_KEY = 'trafficai_officers_v1';
function loadOfficerOverrides() {
try {
const raw = localStorage.getItem(LS_OFFICERS_KEY);
if (!raw) return;
const saved = JSON.parse(raw);
for (const badgeId in saved) {
if (OFFICERS[badgeId]) Object.assign(OFFICERS[badgeId], saved[badgeId]);
else OFFICERS[badgeId] = saved[badgeId];
}
} catch (e) { console.error('Failed to load saved officer data', e); }
}
function saveOfficerOverrides() {
try { localStorage.setItem(LS_OFFICERS_KEY, JSON.stringify(OFFICERS)); }
catch (e) { console.error('Failed to save officer data', e); }
}
loadOfficerOverrides();
function saveState() {
if (!state) return;
try { localStorage.setItem(LS_STATE_KEY, JSON.stringify(state)); }
catch (e) { console.error('Failed to save app state', e); }
}
function reviveDates(obj) {
// timestamps come back as ISO strings after JSON round-trip; feed rendering only
// calls formatTime(new Date(x)), which handles both strings and Date objects fine.
return obj;
}
function loadState() {
try {
const raw = localStorage.getItem(LS_STATE_KEY);
if (!raw) return null;
return reviveDates(JSON.parse(raw));
} catch (e) { console.error('Failed to load saved app state', e); return null; }
}
// These act as links but are <span role="button"> elements (not <a> tags) — some
// mobile browsers/WebViews mishandle javascript: or bare "#" hrefs as app intents
// ("No application can perform this action"), so plain spans + click listeners
// avoid that entirely. This restores Enter/Space keyboard activation for them.
document.querySelectorAll('span[role="button"]').forEach(el => {
el.addEventListener('keydown', e => {
if (e.key === 'Enter' || e.key === ' ') { e.preventDefault(); el.click(); }
});
});
document.getElementById('reset-data-link').addEventListener('click', e => {
e.preventDefault();
if (confirm('This clears all saved violations, fines, and account changes from this browser. Continue?')) {
localStorage.removeItem(LS_STATE_KEY);
localStorage.removeItem(LS_OFFICERS_KEY);
location.reload();
}
});
function showView(id) {
['login-view','forgot-view','register-view','account-view','dashboard-view'].forEach(v => {
document.getElementById(v).classList.toggle('hidden', v !== id);
});
}
// ---------- Login ----------
document.getElementById('login-form').addEventListener('submit', e => {
e.preventDefault();
const badgeId = document.getElementById('badgeId').value.trim();
const password = document.getElementById('password').value;
const officer = OFFICERS[badgeId];
const errBox = document.getElementById('login-error');
if (officer && officer.password === password) {
errBox.classList.add('hidden');
currentOfficer = officer;
enterDashboard();
} else {
errBox.classList.remove('hidden');
}
});
document.getElementById('forgot-link').addEventListener('click', () => {
document.getElementById('forgot-step1-form').classList.remove('hidden');
document.getElementById('forgot-step2-form').classList.add('hidden');
document.getElementById('forgot-otp-box').classList.add('hidden');
document.getElementById('forgot-error').classList.add('hidden');
showView('forgot-view');
});
document.getElementById('back-to-login').addEventListener('click', () => showView('login-view'));
document.getElementById('register-link').addEventListener('click', () => {
document.getElementById('register-form').reset();
document.getElementById('register-error').classList.add('hidden');
document.getElementById('register-success').classList.add('hidden');
showView('register-view');
});
document.getElementById('back-to-login-from-register').addEventListener('click', () => showView('login-view'));
document.getElementById('register-form').addEventListener('submit', e => {
e.preventDefault();
const errBox = document.getElementById('register-error');
```

```
const okBox = document.getElementById('register-success');
errBox.classList.add('hidden');
okBox.classList.add('hidden');
const badgeId = document.getElementById('regBadgeId').value.trim();
const fullName = document.getElementById('regFullName').value.trim();
const rank = document.getElementById('regRank').value;
const station = document.getElementById('regStation').value.trim();
const govEmail = document.getElementById('regGovEmail').value.trim();
const phoneNumber = document.getElementById('regPhoneNumber').value.trim();
const p1 = document.getElementById('regPassword1').value;
const p2 = document.getElementById('regPassword2').value;
if (!badgeId) { errBox.textContent = 'Enter a Badge ID.'; errBox.classList.remove('hidden'); return; }
if (OFFICERS[badgeId]) { errBox.textContent = 'That Badge ID is already registered. Choose another.';
errBox.classList.remove('hidden'); return; }
if (p1.length < 8) { errBox.textContent = 'Password must be at least 8 characters.';
errBox.classList.remove('hidden'); return; }
if (p1 !== p2) { errBox.textContent = 'Passwords do not match.'; errBox.classList.remove('hidden'); return; }
OFFICERS[badgeId] = { badgeId, fullName, rank, station, govEmail, phoneNumber, password: p1 };
saveOfficerOverrides();
okBox.textContent = `Account created for badge ${badgeId}. You can sign in now.`;
okBox.classList.remove('hidden');
document.getElementById('register-form').reset();
setTimeout(() => {
document.getElementById('badgeId').value = badgeId;
showView('login-view');
}, 1200);
});
document.getElementById('forgot-step1-form').addEventListener('submit', e => {
e.preventDefault();
const badgeId = document.getElementById('forgotBadgeId').value.trim();
const officer = OFFICERS[badgeId];
const errBox = document.getElementById('forgot-error');
if (!officer) {
errBox.textContent = 'No officer found with that Badge ID.';
errBox.classList.remove('hidden');
return;
}
errBox.classList.add('hidden');
pendingOtp = String(Math.floor(100000 + Math.random() * 900000));
pendingOtpBadge = badgeId;
const otpBox = document.getElementById('forgot-otp-box');
otpBox.innerHTML = `<strong>Simulated SMS</strong> sent to ${maskPhone(officer.phoneNumber)}.<br>Since this demo
has no real SMS gateway, your code is shown here: <strong>${pendingOtp}</strong>`;
otpBox.classList.remove('hidden');
document.getElementById('forgot-step2-form').classList.remove('hidden');
});
document.getElementById('forgot-step2-form').addEventListener('submit', e => {
e.preventDefault();
const otp = document.getElementById('otpInput').value.trim();
const p1 = document.getElementById('newPassword1').value;
const p2 = document.getElementById('confirmPassword1').value;
const errBox = document.getElementById('forgot-error');
if (otp !== pendingOtp) {
errBox.textContent = 'Incorrect or expired one-time code.';
errBox.classList.remove('hidden');
return;
}
if (p1.length < 8) {
errBox.textContent = 'New password must be at least 8 characters.';
errBox.classList.remove('hidden');
return;
}
if (p1 !== p2) {
errBox.textContent = 'Passwords do not match.';
errBox.classList.remove('hidden');
return;
}
OFFICERS[pendingOtpBadge].password = p1;
pendingOtp = null;
saveOfficerOverrides();
document.getElementById('login-success').textContent = 'Password updated. Sign in with your new password.';
document.getElementById('login-success').classList.remove('hidden');
document.getElementById('login-error').classList.add('hidden');
showView('login-view');
});
// ---------- Account settings ----------
document.getElementById('account-btn').addEventListener('click', () => {
document.getElementById('account-badge-info').textContent = `${currentOfficer.badgeId} ·
${currentOfficer.fullName} · ${currentOfficer.rank}`;
document.getElementById('govEmail').value = currentOfficer.govEmail;
document.getElementById('phoneNumber').value = currentOfficer.phoneNumber;
document.getElementById('contact-success').classList.add('hidden');
document.getElementById('password-success').classList.add('hidden');
document.getElementById('password-error').classList.add('hidden');
document.getElementById('password-form').reset();
showView('account-view');
});
document.getElementById('back-to-dashboard').addEventListener('click', () => showView('dashboard-view'));
document.getElementById('contact-form').addEventListener('submit', e => {
e.preventDefault();
currentOfficer.govEmail = document.getElementById('govEmail').value.trim();
currentOfficer.phoneNumber = document.getElementById('phoneNumber').value.trim();
saveOfficerOverrides();
document.getElementById('contact-success').classList.remove('hidden');
});
document.getElementById('password-form').addEventListener('submit', e => {
e.preventDefault();
```

```
const current = document.getElementById('currentPassword').value;
const p1 = document.getElementById('newPassword2').value;
const p2 = document.getElementById('confirmPassword2').value;
const errBox = document.getElementById('password-error');
const okBox = document.getElementById('password-success');
if (current !== currentOfficer.password) {
errBox.textContent = 'Current password is incorrect.';
errBox.classList.remove('hidden');
okBox.classList.add('hidden');
return;
}
if (p1.length < 8) {
errBox.textContent = 'New password must be at least 8 characters.';
errBox.classList.remove('hidden');
okBox.classList.add('hidden');
return;
}
if (p1 !== p2) {
errBox.textContent = 'Passwords do not match.';
errBox.classList.remove('hidden');
okBox.classList.add('hidden');
return;
}
currentOfficer.password = p1;
errBox.classList.add('hidden');
okBox.classList.remove('hidden');
saveOfficerOverrides();
document.getElementById('password-form').reset();
});
document.getElementById('logout-btn').addEventListener('click', () => {
stopSimulation();
stopCamera();
currentOfficer = null;
document.getElementById('login-form').reset();
document.getElementById('login-success').classList.add('hidden');
document.getElementById('login-error').classList.add('hidden');
showView('login-view');
});
// ============================================================
// Simulation engine (mirrors TrafficSimulationService.java)
// ============================================================
const MAX_FEED_SIZE = 50;
const ALCOHOL_LIMIT_PERCENT = 0.03;
const NOISE_LIMIT_DB = 90.0;
// REPEAT_OFFENDER_THRESHOLD / MULTIPLIER now live on state (editable in UI, see Repeat Offenders panel)
const SIGNAL_JUMP_REPEAT_THRESHOLD = 4;
const SIGNAL_JUMP_REPEAT_MULTIPLIER = 3.0;
const BASE_FINES = {
OVER_SPEED: 1000, SIGNAL_JUMP: 1000, WRONG_LANE: 500, NO_HELMET: 1000,
NO_SEATBELT: 1000, NO_LICENSE: 5000, BLACK_SMOKE_EMISSION: 2000,
NO_NUMBER_PLATE: 5000, DAMAGED_NUMBER_PLATE: 1000, POOR_VEHICLE_CONDITION: 2000,
ALCOHOL_LEVEL_EXCEEDED: 10000, MODIFIED_SILENCER_NOISE: 2000,
};
const twoWheelerChecks = ['NO_HELMET','NO_LICENSE','NO_NUMBER_PLATE','DAMAGED_NUMBER_PLATE','OVER_SPEED','SIGNA
L_JUMP','ALCOHOL_LEVEL_EXCEEDED','MODIFIED_SILENCER_NOISE'];
const fourWheelerChecks = ['NO_SEATBELT','NO_LICENSE','BLACK_SMOKE_EMISSION','NO_NUMBER_PLATE','DAMAGED_NUMBER_
PLATE','POOR_VEHICLE_CONDITION','OVER_SPEED','SIGNAL_JUMP','WRONG_LANE','ALCOHOL_LEVEL_EXCEEDED','MODIFIED_SILE
NCER_NOISE'];
const vehicleTypes = ['Car','Motorbike','Truck','Bus','Auto-rickshaw'];
const locations = ['Main St & 5th Ave','Highway 9 Toll Gate','City Center Junction','Riverside Bridge','School
Zone - Elm St'];
const DUTY_TYPES = ['Patrol','Checkpoint','Traffic Point','Escort','Break','Training'];
const letters = 'ABCDEFGHJKLMNPQRSTUVWXYZ';
let state; // reset on each dashboard entry
function freshState() {
return {
plateOffenseCounts: {},
plateSignalJumpCounts: {},
plateOutstandingFines: {},
vehicleFeed: [],
violationFeed: [],
plateFeed: [],
dutyAlerts: [],
units: [
{ id:'U-001', unitName:'Fixed Cam - Main St', type:'FIXED_CAMERA', connection:'WIRED', location:'Main St & 5th
Ave', status:'ONLINE', batteryPercent:100, lastSync:new Date(), storageDestination:'Secure Gov Drive / Zone-A'
},
{ id:'U-002', unitName:'Fixed Cam - Highway 9', type:'FIXED_CAMERA', connection:'WIRED', location:'Highway 9
Toll Gate', status:'ONLINE', batteryPercent:100, lastSync:new Date(), storageDestination:'Secure Gov Drive /
Zone-A' },
{ id:'U-003', unitName:'Officer Cam - Patrol 12', officerName:'Ranjit Kumar', badgeId:'DL-1042',
type:'OFFICER_CAM', connection:'BLUETOOTH', location:'City Center Junction', status:'ONLINE',
batteryPercent:87, lastSync:new Date(), storageDestination:'Secure Gov Drive / Zone-B',
activityStatus:'ON_DUTY', dutyType:'Patrol' },
{ id:'U-004', unitName:'Officer Cam - Patrol 7', officerName:'Asha Verma', badgeId:'DL-2089',
type:'OFFICER_CAM', connection:'BLUETOOTH', location:'Riverside Bridge', status:'SYNCING', batteryPercent:54,
lastSync:new Date(Date.now()-2*60000), storageDestination:'Secure Gov Drive / Zone-B',
activityStatus:'ON_DUTY', dutyType:'Checkpoint' },
{ id:'U-005', unitName:'Officer Cam - Patrol 3', officerName:'Suresh Rao', badgeId:'DL-3311',
type:'OFFICER_CAM', connection:'WIFI', location:'School Zone - Elm St', status:'OFFLINE', batteryPercent:12,
lastSync:new Date(Date.now()-40*60000), storageDestination:'Secure Gov Drive / Zone-B',
activityStatus:'ON_DUTY', dutyType:'Traffic Point' },
{ id:'U-006', unitName:'Fixed Cam - City Center', type:'FIXED_CAMERA', connection:'WIRED', location:'City Center
Junction', status:'ONLINE', batteryPercent:100, lastSync:new Date(), storageDestination:'Secure Gov Drive /
Zone-A' },
],
totalVehicles: 0,
```

```
totalViolations: 0,
dailyStats: {}, // date string -> { vehicles, violations, byType }
repeatOffenderThreshold: 7,
repeatOffenderMultiplier: 3.0,
fineAmounts: { ...BASE_FINES },
};
}
function rand(n) { return Math.floor(Math.random() * n); }
function randFloat() { return Math.random(); }
function uid() { return Math.random().toString(16).slice(2, 10); }
function todayKey(d = new Date()) { return d.toISOString().slice(0,10); }
function dayAcc(dateKey) {
if (!state.dailyStats[dateKey]) state.dailyStats[dateKey] = { vehicles: 0, violations: 0, byType: {} };
return state.dailyStats[dateKey];
}
function seedPastDays() {
for (let i = 6; i >= 1; i--) {
const d = new Date(Date.now() - i*86400000);
const key = todayKey(d);
const acc = dayAcc(key);
acc.vehicles += 400 + rand(300);
const allTypes = [...new Set([...twoWheelerChecks, ...fourWheelerChecks])];
for (const type of allTypes) {
const count = rand(12);
acc.byType[type] = (acc.byType[type] || 0) + count;
acc.violations += count;
}
}
}
let selectedCameraId = 'auto';
function pickActiveUnit() {
if (selectedCameraId !== 'auto') {
const chosen = state.units.find(u => u.id === selectedCameraId);
// If a specific camera is selected but it's offline, no detections come
// through it (mirrors a real switched-off/disconnected camera).
return (chosen && chosen.status === 'ONLINE') ? chosen : null;
}
const online = state.units.filter(u => u.status === 'ONLINE');
if (online.length === 0) return null;
return online[rand(online.length)];
}
function pushCapped(arr, item) {
arr.unshift(item);
while (arr.length > MAX_FEED_SIZE) arr.pop();
}
function randomPlate() {
return letters[rand(letters.length)] + letters[rand(letters.length)] + ' ' +
(10 + rand(89)) + ' ' +
letters[rand(letters.length)] + letters[rand(letters.length)] +
(1000 + rand(8999));
}
function severityFor(type) {
if (['ALCOHOL_LEVEL_EXCEEDED','NO_HELMET','NO_SEATBELT','OVER_SPEED','SIGNAL_JUMP'].includes(type)) return
'HIGH';
if (['NO_LICENSE','NO_NUMBER_PLATE','POOR_VEHICLE_CONDITION','MODIFIED_SILENCER_NOISE'].includes(type)) return
'MEDIUM';
return 'LOW';
}
function generateVehicleEvent() {
const type = vehicleTypes[rand(vehicleTypes.length)];
const unit = pickActiveUnit();
const location = unit ? unit.location : locations[rand(locations.length)];
const detectedBy = unit ? unit.unitName : 'Unassigned camera';
const speed = Math.round((20 + randFloat()*90) * 10) / 10;
pushCapped(state.vehicleFeed, { id:'V-'+uid(), vehicleType:type, laneOrLocation:location, speedKmh:speed,
detectedByUnit:detectedBy, timestamp:new Date() });
state.totalVehicles++;
dayAcc(todayKey()).vehicles++;
}
function recordViolation(plate, vehicleType, type, location, detectedBy) {
const severity = severityFor(type);
let bacLevel = null, noiseLevelDb = null, signalSecondsLate = null, signalJumpCount = 0;
if (type === 'ALCOHOL_LEVEL_EXCEEDED') bacLevel = Math.round((ALCOHOL_LIMIT_PERCENT + randFloat()*0.07) * 1000)
/ 1000;
if (type === 'MODIFIED_SILENCER_NOISE') noiseLevelDb = Math.round((NOISE_LIMIT_DB + randFloat()*20) * 10) / 10;
if (type === 'SIGNAL_JUMP') {
signalSecondsLate = 1 + rand(10);
state.plateSignalJumpCounts[plate] = (state.plateSignalJumpCounts[plate] || 0) + 1;
signalJumpCount = state.plateSignalJumpCounts[plate];
}
state.plateOffenseCounts[plate] = (state.plateOffenseCounts[plate] || 0) + 1;
const offenseCount = state.plateOffenseCounts[plate];
const generalRepeat = offenseCount > state.repeatOffenderThreshold;
const signalRepeat = type === 'SIGNAL_JUMP' && signalJumpCount > SIGNAL_JUMP_REPEAT_THRESHOLD;
const repeatOffender = generalRepeat || signalRepeat;
let repeatReason = null, multiplier = 1.0;
if (signalRepeat) { repeatReason = `${signalJumpCount} signal jumps on this plate`; multiplier =
SIGNAL_JUMP_REPEAT_MULTIPLIER; }
else if (generalRepeat) { repeatReason = `${offenseCount} total violations on this plate`; multiplier =
state.repeatOffenderMultiplier; }
const baseFine = (state.fineAmounts && state.fineAmounts[type]) ?? BASE_FINES[type] ?? 1000;
const fineAmount = baseFine * multiplier;
state.plateOutstandingFines[plate] = (state.plateOutstandingFines[plate] || 0) + fineAmount;
const violation = {
id:'VIO-'+uid(), type, vehicleType, location, plateNumber:plate, detectedByUnit:detectedBy, severity,
bacLevel, noiseLevelDb, signalSecondsLate, fineAmount, repeatOffender, repeatReason, offenseCount,
timestamp:new Date(), paid:false,
```

```
};
pushCapped(state.violationFeed, violation);
state.totalViolations++;
const acc = dayAcc(todayKey());
acc.violations++;
acc.byType[type] = (acc.byType[type] || 0) + 1;
return violation;
}
function generateViolationAndPlate() {
const plate = randomPlate();
const unit = pickActiveUnit();
const location = unit ? unit.location : locations[rand(locations.length)];
const detectedBy = unit ? unit.unitName : 'Unassigned camera';
const vehicleType = vehicleTypes[rand(vehicleTypes.length)];
pushCapped(state.plateFeed, { id:'P-'+uid(), plateNumber:plate, location, confidence: 80 + randFloat()*19,
detectedByUnit:detectedBy, timestamp:new Date() });
if (rand(3) === 0) {
const isTwoWheeler = vehicleType === 'Motorbike';
const applicable = isTwoWheeler ? twoWheelerChecks : fourWheelerChecks;
const type = applicable[rand(applicable.length)];
recordViolation(plate, vehicleType, type, location, detectedBy);
}
}
function tickFieldUnits() {
for (const u of state.units) {
if (u.type === 'OFFICER_CAM' && u.status === 'ONLINE') {
u.batteryPercent = Math.max(0, u.batteryPercent - rand(2));
u.lastSync = new Date();
if (u.batteryPercent === 0) u.status = 'OFFLINE';
const roll = rand(100);
let newActivity;
if (roll < 72) newActivity = 'ON_DUTY';
else if (roll < 88) newActivity = 'PHONE_DISTRACTION';
else newActivity = 'NOT_STANDING';
u.activityStatus = newActivity;
if (newActivity !== 'ON_DUTY') {
pushCapped(state.dutyAlerts, { id:'DUTY-'+uid(), unitId:u.id, unitName:u.unitName, alertType:newActivity,
location:u.location, timestamp:new Date() });
}
}
}
}
function checkOfficerConduct() {
for (const u of state.units) {
if (u.type !== 'OFFICER_CAM' || u.status !== 'ONLINE') continue;
if (rand(12) === 0) {
pushCapped(state.dutyAlerts, { id:'DUTY-'+uid(), unitId:u.id, unitName:u.unitName,
alertType:'INAPPROPRIATE_LANGUAGE', location:u.location, timestamp:new Date() });
}
}
}
// ============================================================
// Real camera/photo-based detection (TensorFlow.js + COCO-SSD)
// Runs a real pretrained object-detection model on your actual
// webcam feed OR an uploaded photo. Detected cars/trucks/buses/
// motorcycles/persons are genuine model detections (real confidence
// scores, real bounding boxes). Whether a detected vehicle/rider
// gets auto-flagged for OVER_SPEED or NO_HELMET is a simulated
// judgment call (toggle it below) — this demo has no real speed
// radar or helmet-shape classifier, so that specific call is
// randomized, not measured. Plate reads and other checks stay
// fully simulated as before.
// ============================================================
let cocoModel = null;
let cameraStream = null;
let cameraActive = false;
let detectionLoopHandle = null;
const lastDetectionTime = {};
const CAMERA_CLASS_MAP = { car: 'Car', truck: 'Truck', bus: 'Bus', motorcycle: 'Motorbike' };
const DETECTION_COOLDOWN_MS = 4000;
// AI check toggles — when off, live camera / uploaded-photo AI will
// never auto-flag that specific violation type, even if a matching
// vehicle or person is detected in frame. Useful for standing in
// front of the camera yourself without being auto-flagged.
let aiDetectOverspeed = true;
let aiDetectHelmet = true;
const CHECK_FAIL_PROBABILITY = 0.28; // baseline chance any single applicable check fails per detected vehicle
const CHECK_LABELS = {
OVER_SPEED: 'Overspeed', SIGNAL_JUMP: 'Signal Jump', WRONG_LANE: 'Wrong Lane', NO_HELMET: 'Helmet',
NO_SEATBELT: 'Seatbelt', NO_LICENSE: 'License', BLACK_SMOKE_EMISSION: 'Emissions',
NO_NUMBER_PLATE: 'Number Plate', DAMAGED_NUMBER_PLATE: 'Plate Condition', POOR_VEHICLE_CONDITION: 'Vehicle
Condition',
ALCOHOL_LEVEL_EXCEEDED: 'Alcohol', MODIFIED_SILENCER_NOISE: 'Noise Level',
};
function syncToggleLabelState() {
document.getElementById('toggle-overspeed-label').classList.toggle('off', !aiDetectOverspeed);
document.getElementById('toggle-helmet-label').classList.toggle('off', !aiDetectHelmet);
}
document.getElementById('toggle-overspeed').addEventListener('change', e => {
aiDetectOverspeed = e.target.checked;
syncToggleLabelState();
});
document.getElementById('toggle-helmet').addEventListener('change', e => {
aiDetectHelmet = e.target.checked;
syncToggleLabelState();
});
function recordVehicleDetection(vehicleType, confidence, location, detectedByLabel) {
```

```
pushCapped(state.vehicleFeed, {
id: 'V-' + uid(), vehicleType, laneOrLocation: location || 'Live Camera Feed', speedKmh: null,
detectedByUnit: detectedByLabel || `Live Camera AI (${Math.round(confidence * 100)}% conf)`, timestamp: new
Date(),
});
state.totalVehicles++;
dayAcc(todayKey()).vehicles++;
}
// Runs every applicable violation check for a detected vehicle type
// (the full list — helmet, license, plate, seatbelt, emissions,
// alcohol, noise, lane, signal, overspeed — not just one or two).
// The Overspeed AI / Helmet AI toggles can force those two specific
// checks to always pass. Every other check type is always evaluated.
// Returns { results: [{type,label,passed,skipped}], flagged: [violation,...] }.
function runViolationChecklist(vehicleType, location, detectedBy) {
const isTwoWheeler = vehicleType === 'Motorbike';
const applicable = isTwoWheeler ? twoWheelerChecks : fourWheelerChecks;
const results = [];
const flagged = [];
for (const type of applicable) {
if (type === 'OVER_SPEED' && !aiDetectOverspeed) { results.push({ type, label: CHECK_LABELS[type], passed: true,
skipped: true }); continue; }
if (type === 'NO_HELMET' && !aiDetectHelmet) { results.push({ type, label: CHECK_LABELS[type], passed: true,
skipped: true }); continue; }
const failed = randFloat() < CHECK_FAIL_PROBABILITY;
results.push({ type, label: CHECK_LABELS[type], passed: !failed });
if (failed) flagged.push(recordViolation(randomPlate(), vehicleType, type, location, detectedBy));
}
return { results, flagged };
}
// Renders a checklist of PASS/FAIL (true/false) for every check type
// into the given container element, e.g. "Helmet: FAIL License: PASS …"
function renderChecklist(containerEl, vehicleType, results) {
if (!results.length) return;
const parts = results.map(r => {
const cls = r.skipped ? 'skip' : (r.passed ? 'pass' : 'fail');
const val = r.skipped ? 'OFF' : (r.passed ? 'TRUE (pass)' : 'FALSE (violation)');
return `<span class="${cls}">${r.label}: ${val}</span>`;
});
containerEl.innerHTML = `<strong>${vehicleType} — AI check results:</strong><br>` + parts.join('  · 
');
}
async function populateCameraDevices(preselectId) {
const sel = document.getElementById('camera-device-select');
const prior = preselectId != null ? preselectId : sel.value;
try {
if (!navigator.mediaDevices || !navigator.mediaDevices.enumerateDevices) {
sel.innerHTML = '<option value="">Auto (default camera)</option>';
return;
}
let devices = await navigator.mediaDevices.enumerateDevices();
let videoInputs = devices.filter(d => d.kind === 'videoinput');
sel.innerHTML = '<option value="">Auto (default camera)</option>' +
videoInputs.map((d, i) => `<option value="${d.deviceId}">${d.label || ('Camera ' + (i + 1) + ' (grant permission
to see its name)')}</option>`).join('');
if (prior && videoInputs.some(d => d.deviceId === prior)) sel.value = prior;
} catch (e) {
sel.innerHTML = '<option value="">Auto (default camera)</option>';
}
}
populateCameraDevices();
if (navigator.mediaDevices && 'ondevicechange' in navigator.mediaDevices) {
navigator.mediaDevices.addEventListener('devicechange', () => populateCameraDevices());
}
let currentFacingMode = 'environment'; // 'environment' = back camera, 'user' = front/selfie camera
async function openCameraStream() {
const deviceId = document.getElementById('camera-device-select').value;
// Try the exact requested device (built-in laptop cam, external USB cam, phone cam) first.
if (deviceId) {
try { return await navigator.mediaDevices.getUserMedia({ video: { deviceId: { exact: deviceId } } }); }
catch (e) { /* fall through to generic request below */ }
}
try {
return await navigator.mediaDevices.getUserMedia({ video: { facingMode: { ideal: currentFacingMode } } });
} catch (e) {
// Some laptops/external webcams reject facingMode constraints outright — retry with a bare request.
return await navigator.mediaDevices.getUserMedia({ video: true });
}
}
function updateFlipButtonLabel() {
document.getElementById('camera-flip-btn').textContent =
currentFacingMode === 'environment' ? '⇄ Switch to Front' : '⇄ Switch to Back';
}
document.getElementById('camera-flip-btn').addEventListener('click', () => {
currentFacingMode = currentFacingMode === 'environment' ? 'user' : 'environment';
updateFlipButtonLabel();
document.getElementById('camera-device-select').value = ''; // let facingMode pick the camera, not a specific
pinned device
if (cameraActive) {
document.getElementById('camera-status').textContent =
`Switching to ${currentFacingMode === 'user' ? 'front' : 'back'} camera…`;
stopCamera();
startCamera();
}
});
async function startCamera() {
stopVideoAnalysis();
```

```
const statusEl = document.getElementById('camera-status');
const btn = document.getElementById('camera-toggle-btn');
const wrap = document.getElementById('camera-wrap');
wrap.classList.remove('hidden');
// getUserMedia only exists in a "secure context" — https://, or localhost/127.0.0.1.
// Opening this file directly (file://...) or over plain http:// on a phone means
// navigator.mediaDevices won't exist at all, and the camera can never turn on —
// that's a browser security rule, not a bug in this page. Detect it up front and
// say so clearly instead of failing silently.
if (!window.isSecureContext || !navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
statusEl.textContent = 'Camera unavailable: this page must be opened over HTTPS (or localhost) for the browser
to allow camera access — it will not work when opened directly from a file or over plain http://. Host this HTML
file on any HTTPS server (or use "Upload Photo"/"Upload Video" instead, which don\'t need live camera access).';
btn.textContent = 'Enable Camera AI';
document.getElementById('source-badge').textContent = 'SIMULATED';
return;
}
btn.textContent = 'Disable Camera AI';
document.getElementById('source-badge').textContent = 'LIVE CAMERA';
statusEl.textContent = 'Requesting camera access…';
try {
cameraStream = await openCameraStream();
} catch (e) {
statusEl.textContent = 'Camera access denied or unavailable: ' + e.message + '. Check your browser/site camera
permission and that no other app is using the camera.';
stopCamera();
return;
}
populateCameraDevices(document.getElementById('camera-device-select').value); // labels unlock after permission
is granted
const video = document.getElementById('camera-video');
video.srcObject = cameraStream;
statusEl.textContent = 'Camera connected — starting preview…';
try {
await video.play();
} catch (e) {
statusEl.textContent = 'Camera stream connected but the browser blocked autoplay: ' + e.message + '. Tap the
video to start it.';
video.addEventListener('click', () => video.play().catch(() => {}), { once: true });
}
if (!cocoModel) {
statusEl.textContent = 'Loading object-detection model (first time only, a few seconds)…';
try {
cocoModel = await cocoSsd.load();
} catch (e) {
statusEl.textContent = 'Failed to load AI model: ' + e.message;
stopCamera();
return;
}
}
cameraActive = true;
statusEl.textContent = 'Live — real-time vehicle/person detection running on your camera, full violation
checklist active.';
const canvas = document.getElementById('camera-canvas');
const checklistEl = document.getElementById('camera-checklist');
async function detectLoop() {
if (!cameraActive) return;
if (video.readyState >= 2 && video.videoWidth > 0) {
canvas.width = video.videoWidth;
canvas.height = video.videoHeight;
const ctx = canvas.getContext('2d');
ctx.clearRect(0, 0, canvas.width, canvas.height);
// Always-on "LIVE" indicator so it's obvious the camera + AI loop is actually running,
// even in frames where nothing relevant is detected.
ctx.fillStyle = 'rgba(220,53,69,0.85)';
ctx.fillRect(6, 6, 62, 20);
ctx.fillStyle = '#fff';
ctx.font = 'bold 13px monospace';
ctx.fillText('● LIVE', 10, 21);
try {
const predictions = await cocoModel.detect(video);
const unit = pickActiveUnit();
const location = unit ? unit.location : 'Live Camera Feed';
const detectedBy = 'Live Camera AI';
let flaggedThisFrame = null;
for (const p of predictions) {
const mapped = CAMERA_CLASS_MAP[p.class];
const isPerson = p.class === 'person';
const [x, y, w, h] = p.bbox;
let boxColor = (mapped || isPerson) ? '#36d399' : 'rgba(134,148,163,0.5)';
let label = `${p.class} ${Math.round(p.score * 100)}%`;
if (mapped) {
const now = Date.now();
const last = lastDetectionTime[p.class] || 0;
if (now - last > DETECTION_COOLDOWN_MS) {
lastDetectionTime[p.class] = now;
recordVehicleDetection(mapped, p.score, location, `Live Camera AI (${Math.round(p.score * 100)}% conf)`);
const { results, flagged } = runViolationChecklist(mapped, location, detectedBy);
renderChecklist(checklistEl, mapped, results);
if (flagged.length) { boxColor = '#dc3545'; label += ` — ${flagged.length} violation(s)`; flaggedThisFrame =
flagged[0]; }
renderAll();
}
} else if (isPerson) {
const now = Date.now();
const last = lastDetectionTime['person'] || 0;
if (now - last > DETECTION_COOLDOWN_MS) {
```

```
lastDetectionTime['person'] = now;
if (aiDetectHelmet && randFloat() < CHECK_FAIL_PROBABILITY) {
const v = recordViolation(randomPlate(), 'Motorbike', 'NO_HELMET', location, detectedBy);
boxColor = '#dc3545'; label += ' — NO HELMET'; flaggedThisFrame = v; renderAll();
}
}
}
ctx.strokeStyle = boxColor;
ctx.lineWidth = 2;
ctx.strokeRect(x, y, w, h);
ctx.fillStyle = boxColor;
ctx.font = '12px monospace';
ctx.fillText(label, x + 2, y > 12 ? y - 4 : y + 12);
}
if (flaggedThisFrame) jumpToViolation(flaggedThisFrame.id);
} catch (e) { /* ignore transient detection errors */ }
}
detectionLoopHandle = setTimeout(detectLoop, 700);
}
detectLoop();
}
function stopCamera() {
cameraActive = false;
if (detectionLoopHandle) { clearTimeout(detectionLoopHandle); detectionLoopHandle = null; }
if (cameraStream) { cameraStream.getTracks().forEach(t => t.stop()); cameraStream = null; }
document.getElementById('camera-wrap').classList.add('hidden');
document.getElementById('camera-toggle-btn').textContent = 'Enable Camera AI';
document.getElementById('source-badge').textContent = 'SIMULATED';
}
document.getElementById('camera-toggle-btn').addEventListener('click', () => {
if (cameraActive) stopCamera(); else startCamera();
});
document.getElementById('camera-select').addEventListener('change', e => {
selectedCameraId = e.target.value;
updateSourceBadgeForSelection();
});
// Switching the physical device while the camera is already running restarts
// the stream on the newly chosen device (built-in cam, external/USB cam, phone cam).
document.getElementById('camera-device-select').addEventListener('change', () => {
if (cameraActive) { stopCamera(); startCamera(); }
});
document.getElementById('camera-manual-add-btn').addEventListener('click', () => {
const panel = document.getElementById('manual-fine-panel');
panel.classList.remove('hidden');
panel.scrollIntoView({ behavior: 'smooth', block: 'center' });
});
// ---- Photo upload: same real detection model + full checklist, run once on a still image ----
document.getElementById('photo-upload-btn').addEventListener('click', () => {
document.getElementById('photo-upload-input').click();
});
document.getElementById('photo-upload-input').addEventListener('change', async e => {
const file = e.target.files[0];
e.target.value = ''; // allow re-uploading the same file later
if (!file) return;
await analyzePhoto(file);
});
async function analyzePhoto(file) {
const wrap = document.getElementById('photo-wrap');
const statusEl = document.getElementById('photo-status');
const imgEl = document.getElementById('photo-img');
const canvas = document.getElementById('photo-canvas');
const checklistEl = document.getElementById('photo-checklist');
checklistEl.innerHTML = '';
// Uploading a photo reviews a still image — turn off the live camera / video analysis if running.
if (cameraActive) stopCamera();
stopVideoAnalysis();
wrap.classList.remove('hidden');
statusEl.textContent = 'Loading image…';
let dataUrl;
try {
dataUrl = await new Promise((resolve, reject) => {
const reader = new FileReader();
reader.onload = () => resolve(reader.result);
reader.onerror = () => reject(new Error('Could not read file'));
reader.readAsDataURL(file);
});
} catch (e) {
statusEl.textContent = 'Could not read that file: ' + e.message;
return;
}
await new Promise(resolve => {
imgEl.onload = resolve;
imgEl.src = dataUrl;
});
if (!cocoModel) {
statusEl.textContent = 'Loading object-detection model (first time only, a few seconds)…';
try {
cocoModel = await cocoSsd.load();
} catch (e) {
statusEl.textContent = 'Failed to load AI model: ' + e.message;
return;
}
}
statusEl.textContent = 'Analyzing photo — running full violation checklist…';
canvas.width = imgEl.naturalWidth;
canvas.height = imgEl.naturalHeight;
const ctx = canvas.getContext('2d');
```

```
ctx.clearRect(0, 0, canvas.width, canvas.height);
let predictions;
try {
predictions = await cocoModel.detect(imgEl);
} catch (e) {
statusEl.textContent = 'Detection failed: ' + e.message;
return;
}
const unit = pickActiveUnit();
const location = unit ? unit.location : locations[rand(locations.length)];
const detectedBy = 'Uploaded Photo AI';
const allFlagged = [];
const checklistBlocks = [];
let vehicleCount = 0, personCount = 0;
for (const p of predictions) {
const [x, y, w, h] = p.bbox;
const mapped = CAMERA_CLASS_MAP[p.class];
const isPerson = p.class === 'person';
let boxColor = (mapped || isPerson) ? '#36d399' : 'rgba(134,148,163,0.6)';
let label = `${p.class} ${Math.round(p.score * 100)}%`;
if (mapped) {
vehicleCount++;
recordVehicleDetection(mapped, p.score, location, `Uploaded Photo AI (${Math.round(p.score * 100)}% conf)`);
const { results, flagged } = runViolationChecklist(mapped, location, detectedBy);
const parts = results.map(r => {
const cls = r.skipped ? 'skip' : (r.passed ? 'pass' : 'fail');
const val = r.skipped ? 'OFF' : (r.passed ? 'TRUE (pass)' : 'FALSE (violation)');
return `<span class="${cls}">${r.label}: ${val}</span>`;
});
checklistBlocks.push(`<strong>${mapped} #${vehicleCount} — AI check results:</strong><br>` + parts.join('
 ·  '));
if (flagged.length) { boxColor = '#dc3545'; label += ` — ${flagged.length} violation(s)`;
allFlagged.push(...flagged); }
} else if (isPerson) {
personCount++;
if (aiDetectHelmet && randFloat() < CHECK_FAIL_PROBABILITY) {
const v = recordViolation(randomPlate(), 'Motorbike', 'NO_HELMET', location, detectedBy);
boxColor = '#dc3545'; label += ' — NO HELMET'; allFlagged.push(v);
checklistBlocks.push(`<strong>Person #${personCount} — AI check results:</strong><br><span class="fail">Helmet:
FALSE (violation)</span>`);
} else if (aiDetectHelmet) {
checklistBlocks.push(`<strong>Person #${personCount} — AI check results:</strong><br><span class="pass">Helmet:
TRUE (pass)</span>`);
}
}
ctx.strokeStyle = boxColor;
ctx.lineWidth = 2;
ctx.strokeRect(x, y, w, h);
ctx.fillStyle = boxColor;
ctx.font = '13px monospace';
ctx.fillText(label, x + 2, y > 14 ? y - 4 : y + 14);
}
checklistEl.innerHTML = checklistBlocks.join('<br><br>');
renderAll();
const skipped = (!aiDetectOverspeed || !aiDetectHelmet);
if (allFlagged.length) {
const kinds = [...new Set(allFlagged.map(v => v.type.replace(/_/g, ' ')))].join(', ');
statusEl.textContent = `Analysis complete: ${vehicleCount} vehicle(s), ${personCount} person(s) detected.
${allFlagged.length} violation(s) flagged (${kinds}) — opening Violation Alerts…`;
setTimeout(() => jumpToViolation(allFlagged[0].id), 600);
} else {
statusEl.textContent = `Analysis complete: ${vehicleCount} vehicle(s), ${personCount} person(s) detected. No
violations flagged${skipped ? ' — one or both AI toggles are turned off above' : ' this pass — all checks came
back true/pass'}.`;
}
}
// ---- Video upload: live, continuous frame-by-frame detection + full checklist ----
let videoAnalysisActive = false;
let videoDetectionHandle = null;
let uploadedVideoObjectUrl = null;
document.getElementById('video-upload-btn').addEventListener('click', () => {
document.getElementById('video-upload-input').click();
});
document.getElementById('video-upload-input').addEventListener('change', async e => {
const file = e.target.files[0];
e.target.value = '';
if (!file) return;
await analyzeVideo(file);
});
function stopVideoAnalysis() {
videoAnalysisActive = false;
if (videoDetectionHandle) { clearTimeout(videoDetectionHandle); videoDetectionHandle = null; }
if (uploadedVideoObjectUrl) { URL.revokeObjectURL(uploadedVideoObjectUrl); uploadedVideoObjectUrl = null; }
}
async function analyzeVideo(file) {
if (cameraActive) stopCamera();
stopVideoAnalysis();
const wrap = document.getElementById('uploaded-video-wrap');
const statusEl = document.getElementById('uploaded-video-status');
const checklistEl = document.getElementById('uploaded-video-checklist');
const videoEl = document.getElementById('uploaded-video');
const canvas = document.getElementById('uploaded-video-canvas');
checklistEl.innerHTML = '';
wrap.classList.remove('hidden');
statusEl.textContent = 'Loading video…';
uploadedVideoObjectUrl = URL.createObjectURL(file);
videoEl.src = uploadedVideoObjectUrl;
```

```
try {
await new Promise((resolve, reject) => {
videoEl.onloadedmetadata = resolve;
videoEl.onerror = () => reject(new Error('Could not load that video file'));
});
} catch (e) {
statusEl.textContent = e.message;
return;
}
if (!cocoModel) {
statusEl.textContent = 'Loading object-detection model (first time only, a few seconds)…';
try {
cocoModel = await cocoSsd.load();
} catch (e) {
statusEl.textContent = 'Failed to load AI model: ' + e.message;
return;
}
}
try { await videoEl.play(); } catch (e) { /* user can hit the native play control */ }
videoAnalysisActive = true;
statusEl.textContent = 'Live detection running on the uploaded video — full violation checklist active.';
const localLastDetection = {};
async function loop() {
if (!videoAnalysisActive) return;
if (videoEl.ended) {
statusEl.textContent = 'Video ended — analysis stopped. Upload again or scrub back and press play to re-run.';
videoAnalysisActive = false;
return;
}
if (videoEl.paused || videoEl.readyState < 2) {
videoDetectionHandle = setTimeout(loop, 400);
return;
}
canvas.width = videoEl.videoWidth;
canvas.height = videoEl.videoHeight;
const ctx = canvas.getContext('2d');
ctx.clearRect(0, 0, canvas.width, canvas.height);
try {
const predictions = await cocoModel.detect(videoEl);
const unit = pickActiveUnit();
const location = unit ? unit.location : locations[rand(locations.length)];
const detectedBy = 'Uploaded Video AI';
let flaggedThisFrame = null;
for (const p of predictions) {
const mapped = CAMERA_CLASS_MAP[p.class];
const isPerson = p.class === 'person';
const [x, y, w, h] = p.bbox;
let boxColor = (mapped || isPerson) ? '#36d399' : 'rgba(134,148,163,0.5)';
let label = `${p.class} ${Math.round(p.score * 100)}%`;
if (mapped) {
const now = Date.now();
const last = localLastDetection[p.class] || 0;
if (now - last > DETECTION_COOLDOWN_MS) {
localLastDetection[p.class] = now;
recordVehicleDetection(mapped, p.score, location, `Uploaded Video AI (${Math.round(p.score * 100)}% conf)`);
const { results, flagged } = runViolationChecklist(mapped, location, detectedBy);
renderChecklist(checklistEl, mapped, results);
if (flagged.length) { boxColor = '#dc3545'; label += ` — ${flagged.length} violation(s)`; flaggedThisFrame =
flagged[0]; }
renderAll();
}
} else if (isPerson) {
const now = Date.now();
const last = localLastDetection['person'] || 0;
if (now - last > DETECTION_COOLDOWN_MS) {
localLastDetection['person'] = now;
if (aiDetectHelmet && randFloat() < CHECK_FAIL_PROBABILITY) {
const v = recordViolation(randomPlate(), 'Motorbike', 'NO_HELMET', location, detectedBy);
boxColor = '#dc3545'; label += ' — NO HELMET'; flaggedThisFrame = v; renderAll();
}
}
}
ctx.strokeStyle = boxColor;
ctx.lineWidth = 2;
ctx.strokeRect(x, y, w, h);
ctx.fillStyle = boxColor;
ctx.font = '12px monospace';
ctx.fillText(label, x + 2, y > 12 ? y - 4 : y + 12);
}
if (flaggedThisFrame) jumpToViolation(flaggedThisFrame.id);
} catch (e) { /* ignore transient detection errors */ }
videoDetectionHandle = setTimeout(loop, 500);
}
loop();
}
// Scrolls the Violation Alerts panel into view and briefly highlights the
// newly created entry — the closest thing a single-file app has to a
// "redirect" into the violation record it just created.
function jumpToViolation(violationId) {
const item = document.querySelector(`.feed-item[data-vio-id="${violationId}"]`);
const panel = document.getElementById('violation-feed');
(item || panel).scrollIntoView({ behavior: 'smooth', block: 'center' });
if (item) {
item.classList.add('flash-highlight');
setTimeout(() => item.classList.remove('flash-highlight'), 2200);
}
}
```

```
// ---- Editable repeat-offender threshold / multiplier ("> 7 violations = 3x fine") ----
function refreshRepeatBadgeDisplay() {
document.getElementById('repeat-badge-display').innerHTML =
`&gt; ${state.repeatOffenderThreshold} violations = ${state.repeatOffenderMultiplier}&times; fine`;
}
document.getElementById('repeat-edit-btn').addEventListener('click', () => {
document.getElementById('repeat-threshold-input').value = state.repeatOffenderThreshold;
document.getElementById('repeat-multiplier-input').value = state.repeatOffenderMultiplier;
document.getElementById('repeat-badge-display').classList.add('hidden');
document.getElementById('repeat-edit-btn').classList.add('hidden');
document.getElementById('repeat-edit-form').classList.remove('hidden');
});
document.getElementById('repeat-cancel-btn').addEventListener('click', () => {
document.getElementById('repeat-edit-form').classList.add('hidden');
document.getElementById('repeat-badge-display').classList.remove('hidden');
document.getElementById('repeat-edit-btn').classList.remove('hidden');
});
document.getElementById('repeat-save-btn').addEventListener('click', () => {
const t = parseInt(document.getElementById('repeat-threshold-input').value, 10);
const m = parseFloat(document.getElementById('repeat-multiplier-input').value);
if (!Number.isFinite(t) || t < 1 || !Number.isFinite(m) || m < 1) {
alert('Enter a threshold of at least 1 violation and a multiplier of at least 1×.');
return;
}
state.repeatOffenderThreshold = t;
state.repeatOffenderMultiplier = m;
refreshRepeatBadgeDisplay();
document.getElementById('repeat-edit-form').classList.add('hidden');
document.getElementById('repeat-badge-display').classList.remove('hidden');
document.getElementById('repeat-edit-btn').classList.remove('hidden');
renderAll();
saveState();
});
// ---- Editable per-violation-type fine amounts ("even specific vehicle fine") ----
const ALL_VIOLATION_TYPES = [...new Set([...twoWheelerChecks, ...fourWheelerChecks])];
function renderFineAmountsGrid() {
const grid = document.getElementById('fine-amounts-grid');
grid.innerHTML = ALL_VIOLATION_TYPES.map(type => `
<div class="field">
<label for="fine-input-${type}">${type.replace(/_/g, ' ')} (${twoWheelerChecks.includes(type) &&
!fourWheelerChecks.includes(type) ? 'two-wheeler' : (!twoWheelerChecks.includes(type) &&
fourWheelerChecks.includes(type) ? 'four-wheeler' : 'both')})</label>
<input type="number" id="fine-input-${type}" min="0" step="100" value="${state.fineAmounts[type] ??
BASE_FINES[type] ?? 1000}">
</div>
`).join('');
}
document.getElementById('fine-amounts-toggle-btn').addEventListener('click', () => {
const panel = document.getElementById('fine-amounts-panel');
const opening = panel.classList.contains('hidden');
if (opening) renderFineAmountsGrid();
panel.classList.toggle('hidden');
document.getElementById('fine-amounts-msg').textContent = '';
});
document.getElementById('fine-amounts-cancel-btn').addEventListener('click', () => {
document.getElementById('fine-amounts-panel').classList.add('hidden');
});
document.getElementById('fine-amounts-reset-btn').addEventListener('click', () => {
ALL_VIOLATION_TYPES.forEach(type => { document.getElementById(`fine-input-${type}`).value = BASE_FINES[type] ??
1000; });
});
document.getElementById('fine-amounts-save-btn').addEventListener('click', () => {
const updated = {};
for (const type of ALL_VIOLATION_TYPES) {
const val = parseFloat(document.getElementById(`fine-input-${type}`).value);
if (!Number.isFinite(val) || val < 0) {
const msg = document.getElementById('fine-amounts-msg');
msg.textContent = `Enter a valid non-negative amount for ${type.replace(/_/g, ' ')}.`;
msg.className = 'error';
return;
}
updated[type] = val;
}
state.fineAmounts = { ...state.fineAmounts, ...updated };
populateManualFineSelects();
const msg = document.getElementById('fine-amounts-msg');
msg.textContent = 'Fine amounts saved — new violations (manual, camera, photo, video) will use these amounts.';
msg.className = 'success';
renderAll();
saveState();
});
function payFine(id) {
const v = state.violationFeed.find(v => v.id === id && !v.paid);
if (!v) return;
v.paid = true;
state.plateOutstandingFines[v.plateNumber] = Math.max(0, (state.plateOutstandingFines[v.plateNumber] || 0) -
v.fineAmount);
renderViolations();
renderOffenders();
}
window.payFine = payFine;
function getRepeatOffenders() {
const result = Object.keys(state.plateOffenseCounts).map(plate => {
const count = state.plateOffenseCounts[plate];
const outstanding = Math.max(0, state.plateOutstandingFines[plate] || 0);
return { plateNumber: plate, totalViolations: count, outstandingFine: outstanding, repeatOffender: count >
state.repeatOffenderThreshold };
```

```
});
result.sort((a,b) => b.totalViolations - a.totalViolations);
return result;
}
function getDailyStats(days) {
const result = [];
for (let i = days - 1; i >= 0; i--) {
const d = new Date(Date.now() - i*86400000);
const key = todayKey(d);
const acc = state.dailyStats[key];
result.push({ date: key, totalVehicles: acc ? acc.vehicles : 0, totalViolations: acc ? acc.violations : 0 });
}
return result;
}
// ---------- Rendering ----------
function formatTime(d) { return new Date(d).toLocaleTimeString(); }
function renderSummary() {
document.getElementById('stat-vehicles').textContent = state.totalVehicles;
document.getElementById('stat-violations').textContent = state.totalViolations;
const online = state.units.filter(u => u.status === 'ONLINE').length;
document.getElementById('stat-units').textContent = `${online} / ${state.units.length}`;
}
function renderVehicles() {
document.getElementById('stat-plates').textContent = state.plateFeed.length;
const container = document.getElementById('vehicle-feed');
container.innerHTML = state.vehicleFeed.slice(0, 20).map(v => `
<div class="feed-item">
<div class="main">${v.vehicleType} — ${v.laneOrLocation}</div>
<div class="meta">${v.speedKmh != null ? v.speedKmh + ' km/h' : 'real-time
detection'}<br>${v.detectedByUnit}<br>${formatTime(v.timestamp)}</div>
</div>`).join('');
}
function renderViolations() {
const container = document.getElementById('violation-feed');
container.innerHTML = state.violationFeed.slice(0, 20).map(v => `
<div class="feed-item violation severity-${v.severity}" data-vio-id="${v.id}">
<div class="main">
${v.type.replace(/_/g,' ')}${v.bacLevel ? ' (BAC ' + v.bacLevel.toFixed(3) + '%)' : ''}${v.noiseLevelDb ? ' (' +
v.noiseLevelDb.toFixed(1) + ' dB)' : ''}${v.signalSecondsLate ? ' (' + v.signalSecondsLate + 's after red)' :
''} — ${v.vehicleType} ${v.plateNumber}
${v.repeatOffender ? `<span class="repeat-tag" title="${v.repeatReason}">REPEAT · 3&amp;times; FINE</span>` : ''}
</div>
<div class="meta">${v.location} · ${v.detectedByUnit} · ${formatTime(v.timestamp)}${v.repeatOffender ? ' · ' +
v.repeatReason : ''}</div>
<div class="fine-row">
<span class="fine-amount ${v.repeatOffender ? 'repeat' : ''}">Fine: ₹${v.fineAmount.toFixed(0)}</span>
${v.paid ? `<span class="paid-tag">PAID</span>` : `<button class="pay-btn" onclick="payFine('${v.id}')">Mark
fine paid</button>`}
</div>
</div>`).join('');
}
function renderPlates() {
const container = document.getElementById('plate-feed');
container.innerHTML = state.plateFeed.slice(0, 20).map(p => `
<div class="feed-item">
<div class="main">${p.plateNumber}</div>
<div class="meta">${p.confidence.toFixed(1)}% conf<br>${p.detectedByUnit}<br>${formatTime(p.timestamp)}</div>
</div>`).join('');
}
function renderUnits() {
const container = document.getElementById('units-list');
container.innerHTML = state.units.map(u => `
<div class="unit-card">
<div class="unit-top"><span class="unit-name">${u.unitName}</span><span class="unit-status
${u.status}">${u.status}</span></div>
<div class="unit-meta">
Type: <span>${u.type.replace('_',' ')}</span><br>
${u.officerName ? `Officer: <span>${u.officerName} (${u.badgeId})</span><br>` : ''}
Link: <span>${u.connection}</span><br>
Location: <span>${u.location}</span><br>
Last sync: <span>${formatTime(u.lastSync)}</span><br>
Destination: <span>${u.storageDestination}</span>
</div>
${u.type === 'OFFICER_CAM' ? `
<div class="battery-bar"><div class="battery-fill" style="width:${u.batteryPercent}%"></div></div>
<div class="activity-badge activity-${u.activityStatus}">${u.activityStatus.replace('_',' ')}</div>
<div class="duty-type-badge">${u.dutyType || 'Patrol'} duty</div>` : ''}
<div class="unit-card-actions">
<button class="status-toggle-btn ${u.status === 'ONLINE' ? 'is-online' : 'is-offline'}"
onclick="toggleUnitStatus('${u.id}')">
${u.status === 'ONLINE' ? 'Set Offline' : 'Set Online'}
</button>
${u.type === 'OFFICER_CAM' ? `<button class="check-btn" onclick="runBehaviorCheck('${u.id}')" ${u.status !==
'ONLINE' ? 'disabled' : ''}>Run Behavior Check</button>` : ''}
</div>
</div>`).join('');
}
function renderDutyAlerts() {
const container = document.getElementById('duty-feed');
if (state.dutyAlerts.length === 0) {
container.innerHTML = `<div class="empty-note" style="padding:14px 18px;">No duty alerts.</div>`;
return;
}
container.innerHTML = state.dutyAlerts.slice(0, 20).map(d => `
<div class="feed-item duty-${d.alertType}">
<div class="main">${d.unitName} — ${d.alertType.replace(/_/g,' ')}${d.confidence ? ` (${d.confidence}%
confidence)` : ''}</div>
```

```
<div class="meta">${d.location}<br>${formatTime(d.timestamp)}</div>
</div>`).join('');
}
function renderOffenders() {
const list = getRepeatOffenders();
const container = document.getElementById('offenders-table');
if (list.length === 0) {
container.innerHTML = `<div class="empty-note">No offenses recorded yet.</div>`;
return;
}
const rows = list.slice(0, 15).map(o => `
<div class="offender-row ${o.repeatOffender ? 'repeat' : ''}">
<span class="offender-plate">${o.plateNumber}${o.repeatOffender ? '<span class="repeat-tag">REPEAT</span>' :
''}</span>
<span>${o.totalViolations} violation${o.totalViolations === 1 ? '' : 's'}</span>
<span>₹${o.outstandingFine.toFixed(0)} due</span>
<span>${o.repeatOffender ? state.repeatOffenderMultiplier + '× fine rate' : 'Standard rate'}</span>
</div>`).join('');
container.innerHTML = `<div class="offender-row
head"><span>Plate</span><span>Violations</span><span>Outstanding</span><span>Fine rate</span></div>${rows}`;
}
function renderDailyStats() {
const days = getDailyStats(7);
const maxVehicles = Math.max(1, ...days.map(d => d.totalVehicles));
const maxViolations = Math.max(1, ...days.map(d => d.totalViolations));
const today = todayKey();
const container = document.getElementById('trend-chart');
container.innerHTML = days.map(d => {
const vHeight = Math.round((d.totalVehicles / maxVehicles) * 150);
const vioHeight = Math.round((d.totalViolations / maxViolations) * 150);
const isToday = d.date === today;
const label = new Date(d.date + 'T00:00:00').toLocaleDateString(undefined, { weekday: 'short' });
return `
<div class="trend-day">
<div class="trend-bars">
<div class="trend-bar vehicles" style="height:${vHeight}px" title="${d.totalVehicles} vehicles"></div>
<div class="trend-bar violations" style="height:${vioHeight}px" title="${d.totalViolations} violations"></div>
</div>
<div class="trend-label ${isToday ? 'today' : ''}">${isToday ? 'Today' : label}</div>
</div>`;
}).join('');
}
function renderAll() {
renderSummary();
renderVehicles();
renderViolations();
renderPlates();
renderUnits();
renderOffenders();
renderDutyAlerts();
refreshRepeatBadgeDisplay();
// Note: officer roster select controls are refreshed separately (on panel
// open / add / reassign) rather than every renderAll() tick, so an
// in-progress street selection isn't reset mid-choice by the simulation.
saveState();
}
function updateClock() {
document.getElementById('clock').textContent = new Date().toLocaleTimeString();
}
// ---------- Simulation loop management ----------
let simTimers = [];
function startSimulation() {
stopSimulation();
simTimers.push(setInterval(() => { if (!cameraActive) { generateVehicleEvent(); renderAll(); } }, 3000));
simTimers.push(setInterval(() => { generateViolationAndPlate(); renderAll(); }, 7000));
simTimers.push(setInterval(() => { tickFieldUnits(); renderAll(); }, 10000));
simTimers.push(setInterval(() => { checkOfficerConduct(); renderAll(); }, 12000));
simTimers.push(setInterval(renderDailyStats, 15000));
simTimers.push(setInterval(updateClock, 1000));
}
function stopSimulation() {
simTimers.forEach(t => clearInterval(t));
simTimers = [];
}
// ---------- Manual "Issue Fine" entry ----------
function violationOptionsFor(vehicleType) {
return vehicleType === 'Motorbike' ? twoWheelerChecks : fourWheelerChecks;
}
function populateManualFineSelects() {
const vSel = document.getElementById('mf-vehicle-type');
vSel.innerHTML = vehicleTypes.map(t => `<option value="${t}">${t}</option>`).join('');
const lSel = document.getElementById('mf-location');
lSel.innerHTML = locations.map(l => `<option value="${l}">${l}</option>`).join('');
refreshManualFineViolationOptions();
}
function refreshManualFineViolationOptions() {
const vehicleType = document.getElementById('mf-vehicle-type').value;
const tSel = document.getElementById('mf-violation-type');
tSel.innerHTML = violationOptionsFor(vehicleType)
.map(type => `<option value="${type}">${type.replace(/_/g,' ')} — ₹${(state.fineAmounts &&
state.fineAmounts[type]) ?? BASE_FINES[type] ?? 1000}</option>`).join('');
}
populateManualFineSelects();
document.getElementById('mf-vehicle-type').addEventListener('change', refreshManualFineViolationOptions);
function setManualFineMsg(text, kind) {
const el = document.getElementById('manual-fine-msg');
el.textContent = text;
```

```
el.className = kind || '';
}
document.getElementById('manual-fine-toggle-btn').addEventListener('click', () => {
const panel = document.getElementById('manual-fine-panel');
panel.classList.toggle('hidden');
if (!panel.classList.contains('hidden')) {
setManualFineMsg('', '');
document.getElementById('mf-plate').focus();
}
});
document.getElementById('manual-fine-cancel-btn').addEventListener('click', () => {
document.getElementById('manual-fine-panel').classList.add('hidden');
document.getElementById('manual-fine-form').reset();
setManualFineMsg('', '');
});
document.getElementById('manual-fine-form').addEventListener('submit', e => {
e.preventDefault();
const plateRaw = document.getElementById('mf-plate').value.trim();
const plate = plateRaw.toUpperCase().replace(/\s+/g, '');
if (!plate) { setManualFineMsg('Enter a plate number.', 'error'); return; }
if (!state) { setManualFineMsg('Dashboard not ready yet.', 'error'); return; }
const vehicleType = document.getElementById('mf-vehicle-type').value;
const type = document.getElementById('mf-violation-type').value;
const location = document.getElementById('mf-location').value;
const detectedBy = currentOfficer ? `${currentOfficer.fullName} (manual entry)` : 'Manual entry';
const violation = recordViolation(plate, vehicleType, type, location, detectedBy);
renderAll();
setManualFineMsg(`Fine issued: ₹${violation.fineAmount.toFixed(0)} for ${plate}.`, 'success');
document.getElementById('manual-fine-form').reset();
populateManualFineSelects();
});
// ---------- Officer roster: add officer / reassign to another street ----------
function populateOfficerStreetSelect() {
const sel = document.getElementById('off-street');
sel.innerHTML = locations.map(l => `<option value="${l}">${l}</option>`).join('');
}
populateOfficerStreetSelect();
function setAddOfficerMsg(text, kind) {
const el = document.getElementById('add-officer-msg');
el.textContent = text;
el.className = kind || '';
}
function renderOfficerRoster() {
const container = document.getElementById('officer-roster');
if (!container || !state) return;
const officerUnits = state.units.filter(u => u.type === 'OFFICER_CAM');
if (officerUnits.length === 0) {
container.innerHTML = `<div class="empty-note">No officers on patrol yet.</div>`;
return;
}
container.innerHTML = officerUnits.map(u => `
<div class="officer-roster-pair">
<div class="officer-roster-row">
<div class="officer-roster-info">
<strong>${u.officerName || u.unitName}</strong>${u.badgeId ? `<span
class="officer-roster-badge">${u.badgeId}</span>` : ''}
<div class="officer-roster-meta">${u.unitName} · currently on <strong>${u.location}</strong> ·
${u.status}${u.activityStatus ? ' · ' + u.activityStatus.replace(/_/g,' ') : ''} · ${u.dutyType || 'Patrol'}
duty</div>
</div>
<div class="officer-roster-controls">
<select id="street-select-${u.id}">
${locations.map(l => `<option value="${l}" ${l === u.location ? 'selected' : ''}>${l}</option>`).join('')}
</select>
<button class="pay-btn" onclick="reassignOfficerStreet('${u.id}')">Move</button>
</div>
</div>
<div class="officer-roster-row duty-line">
<div class="officer-roster-meta">Duty assignment &amp;amp; conduct check</div>
<div class="officer-roster-controls">
<select id="duty-select-${u.id}">
${DUTY_TYPES.map(d => `<option value="${d}" ${d === (u.dutyType || 'Patrol') ? 'selected' :
''}>${d}</option>`).join('')}
</select>
<button class="pay-btn" onclick="changeOfficerDuty('${u.id}')">Change Duty</button>
<button class="check-btn" onclick="runBehaviorCheck('${u.id}')" ${u.status !== 'ONLINE' ? 'disabled' : ''}>Run
Behavior Check</button>
</div>
</div>
</div>`).join('');
}
function reassignOfficerStreet(unitId) {
const unit = state.units.find(u => u.id === unitId);
if (!unit) return;
const sel = document.getElementById('street-select-' + unitId);
const newStreet = sel.value;
unit.location = newStreet;
renderAll();
renderOfficerRoster();
}
window.reassignOfficerStreet = reassignOfficerStreet;
function changeOfficerDuty(unitId) {
const unit = state.units.find(u => u.id === unitId);
if (!unit) return;
const sel = document.getElementById('duty-select-' + unitId);
const newDuty = sel.value;
unit.dutyType = newDuty;
```

```
const label = unit.officerName ? `${unit.officerName} (${unit.badgeId})` : unit.unitName;
showToast(`  ${label} switched to ${newDuty} duty.`, 'online');
renderAll();
renderOfficerRoster();
}
window.changeOfficerDuty = changeOfficerDuty;
// ---------- Cop behavior detection (on-demand check) ----------
function runBehaviorCheck(unitId) {
const unit = state.units.find(u => u.id === unitId);
if (!unit || unit.status !== 'ONLINE') return;
const label = unit.officerName ? `${unit.officerName} (${unit.badgeId})` : unit.unitName;
const roll = rand(100);
let result, clear;
if (roll < 68) { result = 'ON_DUTY'; clear = true; }
else if (roll < 82) { result = 'PHONE_DISTRACTION'; clear = false; }
else if (roll < 94) { result = 'NOT_STANDING'; clear = false; }
else { result = 'INAPPROPRIATE_LANGUAGE'; clear = false; }
const confidence = Math.round((70 + randFloat() * 29) * 10) / 10;
unit.activityStatus = result === 'INAPPROPRIATE_LANGUAGE' ? 'ON_DUTY' : result;
// INAPPROPRIATE_LANGUAGE is an audio-only signal (not a posture/phone state), so
// it's logged as an alert but doesn't overwrite the video-based activity badge.
pushCapped(state.dutyAlerts, {
id: 'DUTY-' + uid(), unitId: unit.id, unitName: unit.unitName,
officerName: unit.officerName, badgeId: unit.badgeId,
alertType: clear ? 'BEHAVIOR_CHECK_CLEAR' : result,
location: unit.location, confidence, timestamp: new Date(),
});
if (clear) {
showToast(`  Behavior check on ${label}: no issues detected (${confidence}% confidence).`, 'online');
} else {
showToast(`⚠  Behavior check on ${label}: ${result.replace(/_/g,' ').toLowerCase()} detected (${confidence}%
confidence).`, 'offline');
}
renderAll();
renderOfficerRoster();
}
window.runBehaviorCheck = runBehaviorCheck;
document.getElementById('officer-manage-toggle-btn').addEventListener('click', () => {
const panel = document.getElementById('officer-manage-panel');
panel.classList.toggle('hidden');
if (!panel.classList.contains('hidden')) {
setAddOfficerMsg('', '');
renderOfficerRoster();
document.getElementById('off-name').focus();
}
});
document.getElementById('add-officer-form').addEventListener('submit', e => {
e.preventDefault();
if (!state) { setAddOfficerMsg('Dashboard not ready yet.', 'error'); return; }
const officerName = document.getElementById('off-name').value.trim();
const badgeId = document.getElementById('off-badge').value.trim();
const street = document.getElementById('off-street').value;
if (!officerName) { setAddOfficerMsg('Enter the officer\'s name.', 'error'); return; }
if (!badgeId) { setAddOfficerMsg('Enter a badge ID.', 'error'); return; }
if (state.units.some(u => u.badgeId === badgeId)) { setAddOfficerMsg('That badge ID is already on patrol.',
'error'); return; }
const newUnit = {
id: 'U-' + uid(),
unitName: `Officer Cam - ${officerName.split(' ')[0]}`,
officerName, badgeId,
type: 'OFFICER_CAM', connection: 'BLUETOOTH', location: street, status: 'ONLINE',
batteryPercent: 100, lastSync: new Date(), storageDestination: 'Secure Gov Drive / Zone-B',
activityStatus: 'ON_DUTY', dutyType: 'Patrol',
};
state.units.push(newUnit);
renderAll();
renderOfficerRoster();
setAddOfficerMsg(`${officerName} (${badgeId}) added to patrol on ${street}.`, 'success');
document.getElementById('add-officer-form').reset();
populateOfficerStreetSelect();
});
// ---------- Toast notifications ----------
function showToast(message, kind) {
const container = document.getElementById('toast-container');
const toast = document.createElement('div');
toast.className = 'toast' + (kind === 'offline' ? ' offline-toast' : '');
toast.textContent = message;
container.appendChild(toast);
setTimeout(() => {
toast.style.animation = 'toastOut 0.3s ease forwards';
setTimeout(() => toast.remove(), 300);
}, 4200);
}
// ---------- Camera switching ----------
function populateCameraSelect() {
const sel = document.getElementById('camera-select');
const cams = state.units.filter(u => u.type === 'FIXED_CAMERA');
sel.innerHTML = `<option value="auto">Auto (any online camera)</option>` +
cams.map(c => `<option value="${c.id}">${c.unitName} — ${c.location}</option>`).join('');
sel.value = selectedCameraId;
}
function updateSourceBadgeForSelection() {
if (cameraActive) return; // real webcam AI feed already sets its own badge text
const badge = document.getElementById('source-badge');
if (selectedCameraId === 'auto') { badge.textContent = 'SIMULATED'; return; }
const chosen = state.units.find(u => u.id === selectedCameraId);
badge.textContent = (chosen && chosen.status === 'ONLINE') ? 'SIMULATED' : 'CAMERA OFFLINE';
```

```
}
// ---------- Manual unit online/offline toggle ----------
function toggleUnitStatus(unitId) {
const unit = state.units.find(u => u.id === unitId);
if (!unit) return;
const goingOnline = unit.status !== 'ONLINE';
if (goingOnline) {
unit.status = 'ONLINE';
unit.lastSync = new Date();
if (unit.type === 'OFFICER_CAM') {
unit.activityStatus = 'ON_DUTY';
const label = unit.officerName ? `${unit.officerName} (${unit.badgeId})` : unit.unitName;
showToast(`  ${label} is now online and on duty.`, 'online');
pushCapped(state.dutyAlerts, {
id: 'DUTY-' + uid(), unitId: unit.id, unitName: unit.unitName,
officerName: unit.officerName, badgeId: unit.badgeId,
alertType: 'OFFICER_ONLINE', location: unit.location,
timestamp: new Date(),
});
} else {
showToast(`  ${unit.unitName} is now online.`, 'online');
}
} else {
unit.status = 'OFFLINE';
if (unit.type === 'OFFICER_CAM') {
unit.activityStatus = 'OFF_DUTY';
const label = unit.officerName ? `${unit.officerName} (${unit.badgeId})` : unit.unitName;
showToast(`  ${label} has gone offline.`, 'offline');
} else {
showToast(`  ${unit.unitName} has gone offline.`, 'offline');
}
}
renderAll();
renderOfficerRoster();
updateSourceBadgeForSelection();
}
window.toggleUnitStatus = toggleUnitStatus;
function enterDashboard() {
const saved = loadState();
if (saved) {
state = saved;
if (state.repeatOffenderThreshold == null) state.repeatOffenderThreshold = 7;
if (state.repeatOffenderMultiplier == null) state.repeatOffenderMultiplier = 3.0;
if (state.fineAmounts == null) state.fineAmounts = { ...BASE_FINES };
else state.fineAmounts = { ...BASE_FINES, ...state.fineAmounts };
} else {
state = freshState();
seedPastDays();
}
document.getElementById('officer-name').textContent = currentOfficer.fullName;
document.getElementById('officer-badge-rank').textContent = `${currentOfficer.badgeId} ·
${currentOfficer.rank}`;
showView('dashboard-view');
updateClock();
populateCameraSelect();
renderAll();
renderDailyStats();
if (!saved) {
// seed a couple of events immediately so a first-time dashboard isn't empty
generateVehicleEvent(); generateVehicleEvent();
generateViolationAndPlate(); generateViolationAndPlate(); generateViolationAndPlate();
renderAll();
}
startSimulation();
}
</script>
</body>
</html>
```